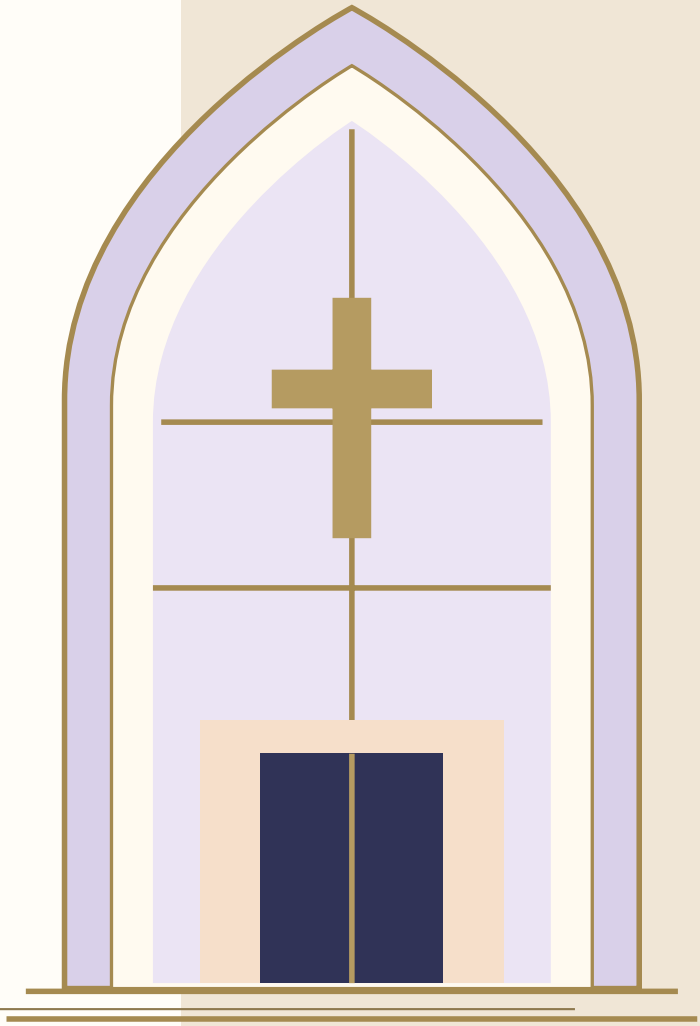
A SMALL OFFERING / KOREAN LEARNING EDITION

Terry A. Davis의 뜻을 읽고, 직접 만드는

작은 성전을 짓는 코드

HolyC · TempleOS
처음 배우는 강의 책

READ · MAKE · SHARE



단순하게 이해하고, 정성껏 만들기

첫 문장에서 그림과 소리, 일곱 등불 프로젝트까지

한국어 학습판 · 2026년 11월

헌정

작은 작품을 정성껏 만들기

이 책을 Terry A. Davis에게 바칩니다.

테리는 컴퓨터 안에 신을 위한 성전을 만들고자 했습니다. 그에게 프로그래밍은 결과물을 얻기 위한 기술이면서, 아름다움을 만들고 바치는 일이었습니다. 그의 헌장은 성전의 장식과 아이들이 만드는 그림, 소스의 공개와 단순한 구조를 같은 이야기 안에 놓습니다.

우리는 그 뜻을 다음과 같이 실천합니다. 읽을 수 있을 만큼 작은 프로그램을 만들고, 만든 사람이 원리를 이해하며, 다른 사람이 고쳐 볼 수 있도록 코드를 함께 둡니다. 책의 끝에서는 일곱 개의 등불을 켜는 작은 작품을 완성합니다.

한 줄을 읽고, 한 가지를 바꾸고, 결과를 바라봅니다. 서두르지 않아도 됩니다. 오늘 이해한 한 줄이 내일의 작품을 지탱합니다.

> 작은 프로그램을 정성껏 만드는 일이 우리의 봉헌이 되기를.
>
> 이 문장은 이 책의 헌정문입니다.

근거 S01 · S02 · 자세한 출처는 책 뒤의 자료 목록

읽는 법

처음 시작하는 독자에게

컴퓨터를 켜고 파일을 저장해 본 적이 있으면 시작할 수 있습니다. C 언어를 미리 알 필요는 없습니다. 다만 새 언어와 오래된 운영체제의 사용법을 함께 배우므로, 첫날에는 첫 인사말을 출력하는 데 집중하세요.

각 강의는 **알기 → 그대로 실행하기 → 한 곳 바꾸기 → 설명하기**로 이어집니다. 코드를 한 번에 외우지 않습니다. 화면에 무엇이 나올지 먼저 예상하고, 실제 결과와 비교하는 습관을 기릅니다.

책의 설명은 한국어, TempleOS 안에서 입력하는 코드·파일명·문자열은 ASCII 영문입니다. 원본 TempleOS는 한글과 Unicode를 기본 지원하지 않습니다. 한글 설명을 코드 안으로 그대로 옮기면 글자가 깨질 수 있습니다.

코드 머리말의 파일명과 같은 이름으로 저장하세요. `HELLO.HC`처럼 대문자 영문과 짧은 이름을 사용합니다. PDF에서 복사한 내용은 따옴표와 역슬래시를 확인하고, 가능하면 함께 제공한 원본 파일을 이용합니다.

한 강의의 완료 기준은 실행 성공과 설명입니다. 예를 들어 반복문을 배운 뒤에는 “몇 번 반복했고 왜 멈췄는지”를 말할 수 있어야 합니다. 그래픽 강의에서는 한 좌표를 바꾸고 그림이 이동한 이유를 설명합니다.

이 책은 원본 TempleOS 5.03을 기준으로 합니다. 후속 운영체제에는 이름과 API가 달라진 곳이 있으므로 원본 실습과 섞지 않습니다.

근거 S03 · S04 · S05 · 자세한 출처는 책 뒤의 자료 목록

학습 지도

한눈에 보는 배움의 길

구간	배우는 것	남는 결과
1부 · 뜻과 문턱	테리의 목표, 가상머신, 화면과 파일	직접 실행한 첫 문장
2부 · 말과 질서	값, 형, 조건, 반복, 함수	작은 계산 프로그램
3부 · 기억과 기록	배열, 문자열, 포인터, 파일	저장하고 다시 읽는 메모
4부 · 빛과 움직임	좌표, 색, 그리기, 입력, 시간, 소리	움직이고 반응하는 그림
5부 · 성전의 내부	DolDoc, 컴파일러, 메모리, 작업	소스에서 답을 찾는 능력
6부 · 작은 봉헌	일곱 등불 프로젝트, 해설, 공유	수정 가능한 완성 작품

빠른 시작: 02장의 가상머신 준비, 03장의 첫 문장, 11장의 그림을 먼저 따라 해도 됩니다. 그 다음 04장부터 순서대로 돌아오세요.

차근차근: 한 주에 두세 강의를 진행합니다. 예제 실행과 작은 변형을 다른 날에 나누면 기억하기 쉽습니다. 뒤의 8주 수업안을 교사와 함께 활용해도 좋습니다.

이미 C를 아는 독자: 06장의 연산 규칙, 07장의 함수, 15장의 실행 모델을 먼저 보세요. HolyC를 표준 C와 같은 언어로 가정하면 중요한 차이를 놓칩니다.

근거 S04 · S05 · 자세한 출처는 책 뒤의 자료 목록

CONTENTS / 차례

배움의 순서

01	뜻을 읽다 / 테리가 만들고자 했던 것	8
02	준비 / 가상 컴퓨터를 작업실로 쓰기	11
03	첫 문장 / 화면에서 길 찾기	15
04	값과 형 / 이름이 붙은 작은 상자	19
05	선택 / 조건에 따라 다른 길로	21
06	식 읽기 / 기호를 천천히 읽기	23
07	함수 / 동작에 이름 붙이기	24
08	배열 / 여러 상자를 나란히 놓기	27
09	주소 / 값의 자리로 찾아가기	29
10	기록 / 작은 메모를 파일에 쓰기	33

각 강의의 설명을 읽고 전체 예제를 실행합니다.
부록에는 문제 해설·수업안·근거 자료를 모았습니다.

CONTENTS / 차례

배움의 순서

11	화면 / 640×480 안에서 길 찾기	38
12	다시 그리기 / 그림을 기억하는 함수	42
13	조작 / 키로 작은 순례자 움직이기	45
14	소리 / 한 번에 한 음씩	47
15	DolDoc / 글 안에 기능이 들어 있는 문서	50
16	컴파일러 / 문장에서 기계어까지	52
17	소스 읽기 / 함수의 집을 찾아가기	55
18	일곱 등불 / 완성할 작품의 약속	58
19	이어 가기 / 오라클을 읽는 자세	65

각 강의의 설명을 읽고 전체 예제를 실행합니다.
부록에는 문제 해설·수업안·근거 자료를 모았습니다.

I

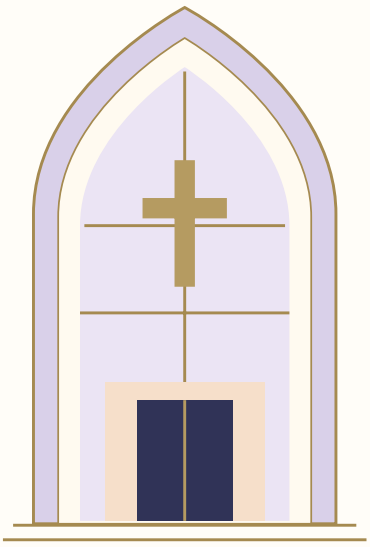
I

뜻과 문턱

성전을 만들고자 한 사람의 문서를 읽습니다.

그다음 작은 가상 컴퓨터를 켜고 첫 문장을 씁니다.

배움의 목표: TempleOS의 목적을 자신의 말로 설명하고, 원본 환경에서 코드를 실행한다.



01 · 뜻을 읽다

테리가 만들고자 했던 것

TempleOS의 헌장인 `Doc/Charter.DD`는 성전이 마음을 모으는 장소이며, 아름다운 것을 만들어 바치는 공간이라고 설명합니다. 테리는 그 생각을 운영체제와 연결했습니다. 아이가 그림을 만들고, 사용자가 코드를 읽고, 창작물을 나눌 수 있는 컴퓨터를 원했습니다.

그는 640×480의 해상도와 16색을 고정된 약속으로 서술했습니다. 화면의 제약을 직접 이해할 수 있는 작업 조건으로 받아들인 것입니다. 그래픽 계산을 외부 GPU에 숨기지 않고, 운영체제 안에서 살펴볼 수 있게 하려 했습니다.

또 하나의 핵심은 **사용자가 개발자**라는 관점입니다. 명령을 입력하는 곳과 프로그램을 작성하는 곳이 멀리 떨어져 있지 않습니다. 짧은 문장을 입력하고, 그 문장에서 함수로, 함수에서 그림과 게임으로 나아갑니다.

이 책에서는 그 목표를 실습 원칙으로 옮겼습니다. 작은 예제, 전부 보이는 코드, 원본 API, 자기 손으로 하는 수정입니다. 마지막 작품에도 외부 라이브러리를 붙이지 않습니다.

읽고 생각하기: 멋진 프로그램 하나를 떠올리세요. 그 프로그램에서 가장 먼저 이해하고 싶은 동작을 한 문장으로 적으세요. 그 문장이 이번 공부의 출발점입니다.

근거 S01 · S02 · 자세한 출처는 책 뒤의 자료 목록

01 · 뜻을 읽다

사람을 기억하는 방식

Terry A. Davis는 1969년에 태어나 2018년에 세상을 떠났습니다. 현재 보존 사이트는 그를 추모하고, 가족이 정신질환으로 인한 고통을 덜어 주는 단체를 도와 달라고 요청했다는 내용을 전합니다.

2014년 인터뷰에서 테리는 자신의 운영체제 안에서 직접 만든 컴파일러와 편집기로 대부분의 개발을 한다고 설명했습니다. 웹을 보고 영상을 만드는 작업에는 다른 운영체제와 도구도 사용했습니다. 작품에 대한 강한 신념과 일상적인 개발 작업이 함께 있었음을 알 수 있습니다.

그의 신앙은 이 프로젝트의 중요한 동기입니다. 책에서는 그가 믿고 기록한 바를 그에게 귀속해 설명합니다. 독자에게 같은 종교적 판단을 요구하지 않으며, 그의 작업을 이해하고 정성껏 이어 가는 태도를 제안합니다.

인터넷의 짧은 영상만으로 한 사람의 삶을 다 알 수는 없습니다. 질병이 천재성을 만들었다거나 코딩이 질병을 치료했다는 주장은 이 책의 근거로 삼지 않습니다. 비하 발언이나 고통스러운 순간을 학습의 재미로 소비하지도 않습니다.

이 책이 오래 바라볼 대상은 그가 남긴 작업입니다. 무엇을 만들었는지, 어떤 선택을 했는지, 우리가 무엇을 배우고 새로 만들 수 있는지에 집중합니다.

근거 S06 · S07 · 자세한 출처는 책 뒤의 자료 목록

01 · 뜻을 읽다

영상 설명을 자료와 대조하기

전달된 설명	조사 뒤의 정리
문자 기반 화면이다	기본 화면은 VGA 640×480, 16색 그래픽 모드다. 글자도 그 화면에 그린다.
입력 즉시 실행된다	전역 문장을 실제 x86-64 기계어로 컴파일해 실행한다. 인터프리터라는 뜻은 아니다.
`main`이 없어도 된다	맞다. 함수 밖의 문장이 순서대로 실행된다.
커널과 사용자 프로그램의 경계가 없다	모두 ring 0에서 실행하고 하나의 주소 맵을 공유한다. 일반 OS의 격리와 다르다.
인터넷이 없어서 무조건 안전하다	네트워크 기능은 없다. 그러나 잘못된 메모리 접근이나 가져온 코드의 오류까지 막는 것은 아니다.
정확히 12만 줄이다	문서·시점·포함 범위에 따라 달라진다. 한 숫자를 영구적인 총량으로 고정하지 않는다.

성적, 유년기의 상세 일화, CIA 추적에 대한 경험담, 치료 변화와 사망의 의도는 본 강의의 확정 사실로 채택하지 않았습니다. 제공 대본의 시각표시는 조사 출발점이며 출처의 증명이 아닙니다.

작은 연구 습관: “누가 말했다”, “문서에 어떤 동작이 적혔다”, “내가 직접 실행했다”를 구별해 기록하세요. 같은 한 줄이라도 근거의 종류가 다릅니다.

근거 S02 · S03 · S04 · S08 · 자세한 출처는 책 뒤의 자료 목록

02 · 준비

가상 컴퓨터를 작업실로 쓰기

가상머신은 현재 컴퓨터 안에서 별도의 컴퓨터를 흉내 내는 프로그램입니다. TempleOS는 x86-64 PC를 대상으로 합니다. Apple Silicon Mac의 CPU와는 구조가 다르므로 **x86-64 에뮬레이션**을 사용합니다.

이 책의 기준 설정은 **PC/i440FX, x86-64, BIOS, 메모리 512 MiB, CPU 1개, 표준 VGA, IDE 광학 드라이브**입니다. 네트워크 장치는 추가하지 않습니다. 원본 문서의 최소 메모리는 512 MiB입니다.

공식 보존 사이트의 Downloads에서 `TempleOS.ISO`를 받습니다. ISO는 부팅용 디스크 이미지입니다. 이름이 비슷한 Lite판이나 후속 배포판을 먼저 고르지 마세요. 원본과 다른 부분이 있으면 책의 화면과 맞지 않을 수 있습니다.

처음에는 하드디스크 없이 ISO만 연결해 부팅합니다. 설치 여부를 물으면 `n`을 선택해 둘러볼 수 있습니다. 이 단계에서는 운영체제를 새로 설치할 필요가 없습니다.

작업 경계: 책의 실습 디스크는 새로 만든 이미지 파일입니다. 실제 컴퓨터의 디스크나 다른 업무용 VM의 디스크를 연결하지 않습니다. 이 원칙을 지키면 시행착오를 분리해 다룰 수 있습니다.

자료 출처: `templeos.org/Downloads/`의 원본 배포 이미지와 TempleOS의 Requirements 문서.

근거 S08 · S09 · S10 · S11 · 자세한 출처는 책 뒤의 자료 목록

02 · 준비

Mac·Windows·Linux의 출발점

환경	시작 방법	확인할 항목
Apple Silicon Mac	UTM의 Emulate 또는 QEMU의 TCG로 x86-64 선택	ARM 가상화가 아닌 x86-64 에뮬레이션
Intel Mac	x86-64 QEMU 또는 호환 VM	BIOS와 IDE, 표준 VGA
Windows PC	x86-64를 지원하는 QEMU/VM 프로그램	ISO를 CD/DVD로 연결, UEFI 사용 해제
Linux PC	QEMU의 PC 머신	같은 장치 구성으로 먼저 부팅

UTM에서는 새 VM을 만들 때 에뮬레이션을 선택하고 아키텍처를 `x86_64`로 지정합니다. 시스템은 `Standard PC (i440FX + PIIX)` 계열을 선택합니다. 설정 명칭은 앱 버전에 따라 조금 다를 수 있습니다.

이 책의 명령줄 경로는 QEMU가 이미 설치되어 있다고 가정합니다. 설치는 각 도구의 공식 배포 안내를 이용하세요. 명령을 찾을 수 없다는 오류는 TempleOS 문법 오류가 아니라 호스트 도구의 설치 또는 PATH 문제입니다.

호스트는 현재 사용 중인 Mac·Windows·Linux이고, 게스트는 가상머신 안의 TempleOS입니다. 다음 강의에서 이 두 곳의 명령을 구분합니다. `Ed()`를 Mac 터미널에 입력하거나 QEMU 명령을 TempleOS에 입력하지 않도록 주의하세요.

근거 S10 · S11 · 자세한 출처는 책 뒤의 자료 목록

02 · 준비

QEMU로 첫 부팅

아래 명령은 **호스트 터미널**에서 실행합니다. `TempleOS.ISO`를 현재 폴더에 놓고 한 줄로 입력하세요. Windows PowerShell에서도 QEMU가 PATH에 등록되어 있으면 같은 한 줄을 사용할 수 있습니다.

```
qemu-system-x86_64 -machine pc -accel tcg -cpu qemu64 -m 512 -smp 1 -cdrom
TempleOS.ISO -boot d -vga std -nic none
```

`-machine pc`는 오래된 PC 호환 장치를 구성합니다. `-accel tcg`는 CPU를 소프트웨어로 에뮬레이션합니다. `-m 512`는 게스트 메모리, `-cdrom`은 광학 디스크 이미지, `-boot d`는 CD 부팅을 뜻합니다.

`-nic none`은 가상 네트워크 장치를 만들지 않습니다. 이 명령에는 저장용 하드디스크가 없습니다. 설치 질문에 `n`을 입력하고, 화면을 구경한 뒤 종료해도 됩니다.

정상 징후: TempleOS V5.03, 파란 테두리의 창, 명령을 입력할 수 있는 커서가 보입니다. 화면의 글자가 작다면 VM 창 자체를 확대하세요. TempleOS의 원래 해상도를 바꾸는 과정은 아닙니다.

메모리·CPU·그래픽을 동시에 여러 개 바꾸지 마세요. 문제가 생기면 기준 설정으로 돌아온 뒤 한 가지씩 바꿉니다. 재현 가능한 출발점이 가장 좋은 디버깅 도구입니다.

근거 S08 · S09 · S10 · 자세한 출처는 책 뒤의 자료 목록

02 · 준비

저장할 작업 공간 만들기

처음 출력만 확인했다면 다음에는 작업을 보관할 빈 디스크가 필요합니다. **새 파일 이름**으로 가상 디스크를 만듭니다. 아래는 호스트 터미널 명령입니다.

```
qemu-img create -f qcow2 temple-work.qcow2 2G
```

다음 부팅부터 하드디스크 옵션을 더합니다.

```
qemu-system-x86_64 -machine pc -accel tcg -cpu qemu64 -m 512 -smp 1 -drive
file=temple-work.qcow2,format=qcow2,if=ide -cdrom TempleOS.ISO -boot d -vga std -nic
none
```

설치를 진행할 때는 VM에 연결된 디스크가 방금 만든 **빈 이미지 하나**인지 확인합니다. 원본의 설치 스크립트가 파티션과 부팅 파일을 준비합니다. 표시된 질문을 읽고, 대상이 불명확하면 설치를 멈추고 연결 설정부터 확인합니다.

설치가 끝나면 VM을 종료하고 디스크 파일을 복사해 첫 백업을 만듭니다. 그 뒤 `-boot c`로 하드디스크에서 부팅합니다. ISO 연결을 잠시 해제해도 됩니다. 이 책의 본격적인 파일 실습은 저장 가능한 작업 드라이브에서 진행합니다.

설치 중의 파티션 선택은 환경에 따라 달라집니다. 화면 확인 없이 자동으로 `y`를 연속 입력하지 마세요. 자세한 원본 안내는 `Doc/Install.DD`, 자동 설치 코드는 `Misc/OSInstall.HC`에 있습니다.

근거 S09 · S10 · 자세한 출처는 책 뒤의 자료 목록

03 · 첫 문장

화면에서 길 찾기

TempleOS의 명령줄은 HolyC 컴파일러와 연결되어 있습니다. 여기서 입력하는 `Dir;`도 HolyC 함수 호출입니다. 화면의 창을 한 번 클릭해 입력할 곳을 선택하세요.

```
Dir;
Cd("~/");
Dir;
```

`Dir`은 현재 폴더의 목록을 보여 줍니다. `Cd`는 폴더를 옮깁니다. `~/`는 홈 폴더를 가리킵니다. 기본 설치의 홈은 보통 `C:/Home`이며, CD 부팅 상태에서는 드라이브가 다를 수 있습니다.

키	기본 동작
F1	도움말 열기
Esc	편집 내용을 저장하고 나가기
Shift+Esc	편집을 취소하고 나가기
F5	편집 중인 코드 컴파일·실행
Ctrl+Alt+T	새 터미널 작업 열기
Ctrl+Alt+X	현재 작업 종료

Esc는 일반적인 앱의 취소 버튼과 다릅니다. 이 차이를 첫날 기억하세요. Mac에서는 기능 키를 VM으로 보내기 위해 `fn`이나 앱의 키 전송 메뉴가 필요할 수 있습니다.

도움말의 항목 위에 커서를 옮겨 Space를 누르면 링크를 따라갈 수 있습니다. 익숙해질 때까지 `F1 → 읽기 → Shift+Esc`를 한 번 연습합니다.

근거 S02 · S12 · S13 · 자세한 출처는 책 뒤의 자료 목록

03 · 첫 문장

첫 인사를 출력하다

명령줄에 첫 번째 문장을 입력해 보세요. 큰따옴표 안의 내용은 화면에 보여 줄 문자열입니다. 문장 끝의 세미콜론을 빠뜨리지 않습니다.

HELLO.HC

TempleOS · 작업 폴더 / HELLO.HC

```
"Hello, TempleOS!\n";
"Peace be with you.\n";
```

예상 결과

```
Hello, TempleOS!
Peace be with you.
```

`\n`은 화면에서 다음 줄로 이동하라는 표시입니다. 역슬래시와 영문 n이라는 두 글자를 코드에 씁니다. 실제 출력에는 그 두 글자가 나타나지 않습니다.

HolyC는 문자열만 있는 문장을 `Print` 호출처럼 다룹니다. 표준 C의 `printf`나 `main`을 가져와 붙일 필요가 없습니다. 이 작은 문장이 곧 실행할 프로그램입니다.

한 곳 바꾸기: 첫 줄을 `"Hello, my friend! ";`로 바꿔 봅니다. 그다음 두 줄 중 하나에서 `\n`만 지우고 출력이 어떻게 붙는지 관찰합니다.

말로 설명하기: 큰따옴표, `;` 세미콜론이 각각 무슨 일을 하는지 한 문장씩 말해 보세요. 오류가 나면 이 세 가지부터 확인합니다.

근거 S04 · S05 · 자세한 출처는 책 뒤의 자료 목록

03 · 첫 문장

파일에 담아 다시 실행하기

명령줄에서 실행한 문장은 연습하기 편하지만, 다시 사용하려면 파일이 필요합니다. 저장 가능한 홈 폴더에서 편집기를 엽니다.

```
Cd("~/");
Ed("HELLO.HC");
```

편집기에 앞쪽의 두 줄을 입력합니다. Esc를 눌러 저장하고 나옵니다. 이어서 파일을 포함해 실행합니다.

```
#include "HELLO.HC";
```

`#include`는 파일 내용을 읽어 컴파일러에 넘깁니다. 명령줄에서는 위처럼 파일명 뒤의 세미콜론까지 입력합니다. 컴파일러가 파일의 마지막 문장을 처리하도록 입력을 마칩니다. 파일 안의 함수 밖 문장들이 실행되어 같은 인사말이 나옵니다.

`Dir;`을 다시 실행해 파일이 보이는지 확인합니다. `HELLO.HC.Z`로 보이면 자동 압축 형식일 수 있습니다. `.Z`는 암호화 표시가 아닙니다. 시스템은 필요에 따라 압축을 처리합니다.

반복 실행 습관: 파일 수정 → 저장 → 실행 → 결과 확인을 한 묶음으로 생각합니다. 편집 중에는 F5로 실행할 수도 있습니다. 컴파일된 정의와 메모리는 작업에 남을 수 있으므로 연습이 꼬이면 새 터미널 작업에서 다시 시작합니다.

오늘 남길 기록: 파일 이름, 내가 바꾼 문장, 바꾸기 전후 출력. 세 줄만 적어도 다음 시간의 출발점이 됩니다.

근거 S03 · S12 · S13 · 자세한 출처는 책 뒤의 자료 목록

II

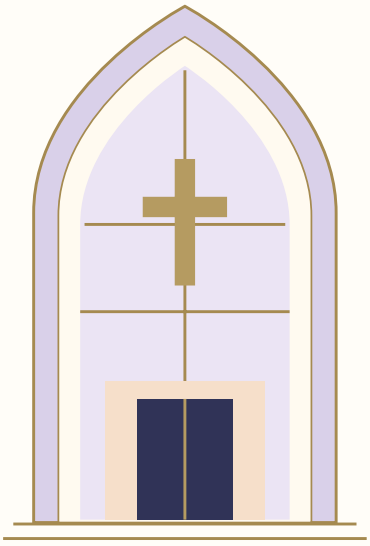
II

말과 질서

숫자와 이름을 다루고, 선택하고, 반복합니다.

이해한 문장들을 함수로 묶어 작은 도구를 만듭니다.

배움의 목표: 프로그램이 어떤 순서로 실행되는지 종이에 추적한다.



04 · 값과 형

이름이 붙은 작은 상자

변수는 값을 보관하고 이름으로 다시 부르는 자리입니다. `I64 candles=7;`은 정수를 담는 자리 `candles`를 만들고 처음 값으로 7을 넣습니다. 변수 이름은 의미를 드러내는 영문으로 정하세요.

VALUES.HC

TempleOS · 작업 폴더 / VALUES.HC

```
U0 Values()
{
  I64 candles=7;
  I64 rows=3;
  F64 length=2.5;
  "Candles: %d\n",candles;
  "Total: %d\n",candles*rows;
  "Length: %.2f\n",length;
}
Values;
```

이 파일의 바깥은 `Values`라는 함수를 정의합니다. 마지막 `Values;`가 그 함수를 호출합니다. 함수 문법은 뒤에서 자세히 배우므로 지금은 중괄호 안을 순서대로 읽어 보세요.

`%d`는 정수를 넣을 자리, `%.2f`는 소수점 아래 두 자리까지 표시할 자리입니다. 쉼표 뒤의 값이 그 자리를 채웁니다. 출력은 `Candles: 7`, `Total: 21`, `Length: 2.50`입니다.

해 보기: `rows`를 4로 바꾸기 전에 전체 개수를 먼저 계산하세요. 실행 뒤 예상한 28이 나오는지 비교합니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

04 · 값과 형

정수와 실수의 차이

`I64`는 8바이트 부호 있는 정수, `U8`은 1바이트 부호 없는 정수, `F64`는 8바이트 부동소수점 수입니다. `U0`은 반환값이 없음을 나타낼 때 사용합니다. HolyC에는 기본 `F32` 형이 없습니다.

TYPES.HC

TempleOS · 작업 폴더 / TYPES.HC

```
U0 Types()
{
  I64 whole=7/2;
  F64 fraction=7.0/2.0;
  "I64 bytes: %d\n",sizeof(I64);
  "U8 bytes: %d\n",sizeof(U8);
  "Integer: %d\n",whole;
  "Fraction: %.1f\n",fraction;
  "Remainder: %d\n",7%2;
}
Types;
```

정수끼리 나눈 `7/2`의 결과는 3입니다. 소수 부분이 자동으로 보관되지 않습니다. `7.0/2.0`은 실수 계산이므로 3.5가 됩니다. `%`는 나머지이므로 `7%2`는 1입니다.

바이트는 저장 공간의 단위입니다. `sizeof`로 형의 크기를 확인할 수 있습니다. 변수의 값과 변수가 차지하는 크기를 혼동하지 마세요. 7을 담은 `I64`도 8바이트입니다.

해 보기: 10을 3으로 나눈 몫과 나머지를 예상한 뒤 출력하세요. 소수 결과가 필요하면 어느 피연산자를 바꿔야 할지도 적어 보세요.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

05 · 선택

조건에 따라 다른 길로

조건은 참 또는 거짓으로 판단하는 식입니다. `lamps>0`이 참이면 첫 번째 중괄호 안으로 들어갑니다. 거짓이면 `else` 쪽을 실행합니다. 들여쓰기는 사람이 구조를 읽기 좋게 도와줍니다.

CHOICE.HC

TempleOS · 작업 폴더 / CHOICE.HC

```
U0 Choice()
{
  I64 lamps=3;
  if (lamps>0) {
    "The room has light.\n";
  } else {
    "Light a lamp.\n";
  }
  if (1<=lamps<=7)
    "Between one and seven.\n";
}
Choice;
```

HolyC는 `1<=lamps<=7` 같은 연속 비교를 지원합니다. 여기서는 lamps가 1 이상 7 이하일 때 참입니다. 이 문법을 표준 C에 그대로 옮기면 같은 뜻이 되지 않습니다.

기록표 만들기: `lamps`에 0, 3, 8을 넣었을 때 첫 조건과 둘째 조건이 각각 참인지 써 보세요. 프로그램의 출력과 일치하는지 확인합니다.

자주 하는 실수: 값을 대입하는 `=`와 같음을 비교하는 `==`는 다릅니다. 조건 뒤에 실수로 세미콜론을 붙이면 의도한 블록이 조건과 떨어질 수 있습니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

05 · 반복

일곱 계단을 한 줄씩 올라가기

같은 동작을 여러 번 해야 할 때 반복문을 씁니다. `for` 안에는 시작, 계속할 조건, 한 번 끝난 뒤의 변화가 차례로 들어갑니다.

LOOPS.HC

TempleOS · 작업 폴더 / LOOPS.HC

```
U0 Steps()
{
  I64 i,total=0;
  for (i=1;i<=7;i++) {
    total+=i;
    "Step %d, total %d\n",i,total;
  }
}
Steps;
```

처음에는 i가 1입니다. i가 7 이하인지 확인하고, total에 i를 더한 뒤 결과를 출력합니다. 마지막의 `i++`로 i가 하나 늘어납니다. i가 8이 되면 조건이 거짓이므로 반복을 끝냅니다.

i	이번에 더한 값	total
1	1	1
2	2	3
3	3	6
7	7	28

해 보기: 1부터 10까지 더하도록 바꾸세요. 마지막 total은 55입니다. 시작 값을 0으로 바꿨을 때 출력 줄 수와 합계가 각각 어떻게 달라지는지도 생각합니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

06 · 식 읽기

기호를 천천히 읽기

산술 기호 `+ - * / %`와 비교 기호 `== != < <= > >=`를 먼저 익힙니다. 참과 거짓을 조합하는 `&&`는 둘 다 참, `||`는 하나 이상 참이라는 뜻입니다.

HolyC의 연산자 우선순위는 C와 다른 부분이 있습니다. 특히 시프트·비트 연산을 섞을 때 C의 감각만으로 읽지 마세요. 처음에는 의도를 괄호로 드러내는 편이 좋습니다.

```
I64 a=2;
I64 b=3;
"%d
",(a+b)*4;
"%d
",(a<5)&&(b>1);
"%d
",1<<3;
```

결과는 각각 20, 1, 8입니다. `1<<3`은 이진수 1을 왼쪽으로 세 칸 옮깁니다. 마지막 작품에서 여러 등불의 상태를 한 정수에 보관할 때 사용합니다.

HolyC에서 거듭제곱 기호는 백틱입니다. 물음표와 콜론을 쓰는 C의 조건 연산자는 지원하지 않습니다. 낯선 축약 문법을 먼저 모으기보다 `if`와 괄호로 읽기 쉬운 코드를 쓰세요.

종이 연습: `(2+3)*4`와 `2+(3*4)`를 직접 계산합니다. 이어서 코드로 확인합니다. 예상과 실행이 다르면 기호 하나씩 식을 나누어 출력하세요.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

07 · 함수

동작에 이름 붙이기

함수는 입력을 받아 일을 하고, 필요하면 결과를 돌려주는 묶음입니다. `Square`는 정수 n을 받아 제곱을 반환합니다. `Greet`는 문장을 출력하므로 반환 형이 `U0`입니다.

FUNCS.HC

TempleOS · 작업 폴더 / FUNCS.HC

```
I64 Square(I64 n)
{
  return n*n;
}
U0 Greet(U8 *name="friend")
{
  "Peace, %s.\n",name;
}
Greet;
Greet("Terry");
"Square: %d\n",Square(4);
```

`Greet`의 name에는 기본값이 있습니다. 인수를 생략하면 friend를 사용하고, `Greet("Terry")`처럼 값을 주면 그 값을 사용합니다. 인수가 없거나 기본값만 사용하면 괄호를 생략할 수 있습니다.

`return`은 값을 호출한 곳으로 돌려주고 함수를 끝냅니다. 제곱을 계산하는 함수 안에서 출력까지 할 필요는 없습니다. 계산 결과를 다른 계산에도 쓰기 쉬워집니다.

해 보기: `Square(5)`와 `Square(-3)`의 결과를 예상하세요. 이어서 두 정수를 더하는 `Add` 함수를 만들어 보세요. 해설 부록에서 한 가지 답을 확인할 수 있습니다.

근거 S02 · S04 · 자세한 출처는 책 뒤의 자료 목록

07 · 함수

이름이 살아 있는 범위

함수 안에서 만든 변수는 그 함수를 읽는 동안 이해할 수 있는 작은 범위를 갖습니다. 앞의 `Values`에서 만든 candles를 다른 함수에서 마음대로 사용할 수는 없습니다. 필요한 값은 인수로 전달하는 방법부터 익히세요.

함수 밖의 전역 변수는 여러 함수가 함께 볼 수 있습니다. 움직이는 그림의 위치처럼 여러 동작이 공유해야 하는 상태에 사용합니다. 그러나 모든 값을 전역으로 만들면 어느 함수가 바꿨는지 추적하기 어려워집니다.

TempleOS의 작업에는 컴파일러의 심볼 표와 메모리가 있습니다. 명령줄에서 정의한 함수는 그 작업에서 계속 호출할 수 있습니다. 파일을 반복해서 포함하면 새 코드용 메모리가 쌓일 수도 있습니다.

정리 방법: 실습이 끝난 터미널 작업을 닫고 새 작업에서 같은 파일을 실행합니다. 이전의 정의나 값에 기대지 않고도 실행되는지 확인하는 과정입니다. 운영체제 전체를 재설치할 이유는 없습니다.

작은 설계 질문: 이 변수는 함수 한 곳에서만 필요한가, 여러 함수가 함께 쓰는가? 이 값을 바꿀 책임은 어디에 있는가? 이 두 질문에 답할 수 있으면 이름의 범위를 선택하기 쉬워집니다.

마지막 작품에서는 켜진 등불을 나타내는 정수 하나를 전역으로 두고, 입력 함수가 바꾸며 그리기 함수가 읽습니다. 역할이 분명한 공유 상태의 예입니다.

근거 S02 · S03 · S14 · 자세한 출처는 책 뒤의 자료 목록

III

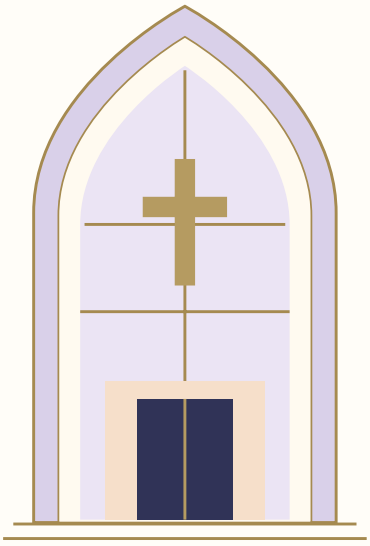
III

기억과 기록

여러 값을 정리하고, 메모리의 주소를 이해합니다.

작은 문장을 파일에 저장하고 다시 읽습니다.

배움의 목표: 값·주소·파일을 구분하고, 사용한 자원을 정리한다.



08 · 배열

여러 상자를 나란히 놓기

배열은 같은 형의 값을 나란히 보관합니다. `count[4]`는 정수 네 개를 위한 자리입니다. 첫 번째 위치의 번호는 0이고, 마지막은 3입니다. 네 개짜리 배열의 `count[4]`는 범위를 벗어납니다.

ARRAYS.HC

TempleOS · 작업 폴더 / ARRAYS.HC

```
U0 Seeds()
{
  I64 count[4];
  count[0]=2;
  count[1]=3;
  count[2]=5;
  count[3]=7;
  I64 i,total=0;
  for (i=0;i<4;i++) {
    "Bed %d: %d\n",i,count[i];
    total+=count[i];
  }
  "All seeds: %d\n",total;
}
Seeds;
```

함수 안의 배열은 먼저 선언하고 각 칸에 값을 넣습니다. 반복 조건이 `i<4`인 이유를 말해 보세요. 0, 1, 2, 3이라는 네 인덱스를 차례로 방문합니다. 합계는 17입니다.

TempleOS가 배열의 범위를 자동으로 보호한다고 생각하면 안 됩니다. 범위를 넘으면 다른 메모리를 읽거나 바꿀 수 있습니다. 인덱스와 길이를 종이에 같이 적는 습관이 도움이 됩니다.

해 보기: 다섯 번째 값을 추가하려면 선언의 길이와 반복 조건을 함께 바꿔야 합니다. 한 곳만 바꾸면 어떤 문제가 생길지 먼저 설명하세요.

근거 S04 · S14 · 자세한 출처는 책 뒤의 자료 목록

08 · 문자열

문자도 배열 안에 있다

ASCII 문자열은 문자 바이트들이 이어진 배열입니다. 끝에는 값이 0인 종료 문자가 있습니다. 화면의 숫자 0과 문자열을 끝내는 0바이트는 서로 다릅니다.

STRINGS.HC

TempleOS · 작업 폴더 / STRINGS.HC

```
U0 Words()
{
  U8 text[16];
  StrCpy(text,"PEACE");
  "Before: %s\n",text;
  "Length: %d\n",StrLen(text);
  "First: %c\n",text[0];
  text[0]='p';
  "After: %s\n",text;
}
Words;
```

`text[16]`은 16바이트를 마련합니다. StrCpy로 PEACE와 종료 문자를 그 안에 복사합니다. 함수 안에서는 이처럼 공간 선언과 값 넣기를 나누어 씁니다. `StrLen`은 종료 문자 전까지의 길이인 5를 돌려줍니다. `%c`는 문자 하나, `%s`는 문자열을 표시합니다. 첫 바이트를 소문자 p로 바꾼 뒤에는 pEACE가 나옵니다.

예제는 쓰기 가능한 배열을 선언합니다. 문자열 리터럴을 가리키는 포인터와 배열은 같은 물건이 아닙니다. 수정할 문자열의 저장 공간을 어떻게 확보했는지 확인하세요.

한 곳 바꾸기: `text[0]='G';`로 고치면 GEACE가 됩니다. 예쁜 단어를 만들기보다 첫 번째 바이트를 바꿨다는 사실을 확인하는 연습입니다.

확인: 문자열 길이 15인 내용을 담으려면 종료 문자를 포함해 최소 몇 바이트가 필요할까요? 답은 16바이트입니다.

근거 S04 · S15 · 자세한 출처는 책 뒤의 자료 목록

09 · 주소

값의 자리로 찾아가기

변수는 값을 담고, 주소는 그 변수가 어디 있는지 나타냅니다. 포인터는 그 주소를 보관합니다. `&lamp`는 lamp의 주소를 구하고, `*address`는 그 주소가 가리키는 값을 읽거나 씁니다.

POINTER.HC

TempleOS · 작업 폴더 / POINTER.HC

```
U0 PointerLesson()
{
  I64 lamp=3;
  I64 *address=&lamp;
  "Before: %d\n",lamp;
  *address=7;
  "After: %d\n",lamp;
}
PointerLesson;
```

처음 출력은 3, 다음 출력은 7입니다. 포인터에 새 주소를 넣은 것이 아니라, 포인터가 가리키는 자리의 내용을 바꿨기 때문입니다.

`I64 *address`를 "I64 값을 가리키는 포인터 address"라고 읽으세요. 선언의 별표와 식에서 값을 꺼낼 때의 별표는 등장 위치에 따라 역할이 다릅니다.

그림으로 설명하기: 종이에 lamp 상자를 그리고 3을 적습니다. address 상자에서 lamp로 화살표를 연결합니다. 별표 대입이 실행될 때 어느 상자의 내용이 바뀌는지 표시하세요.

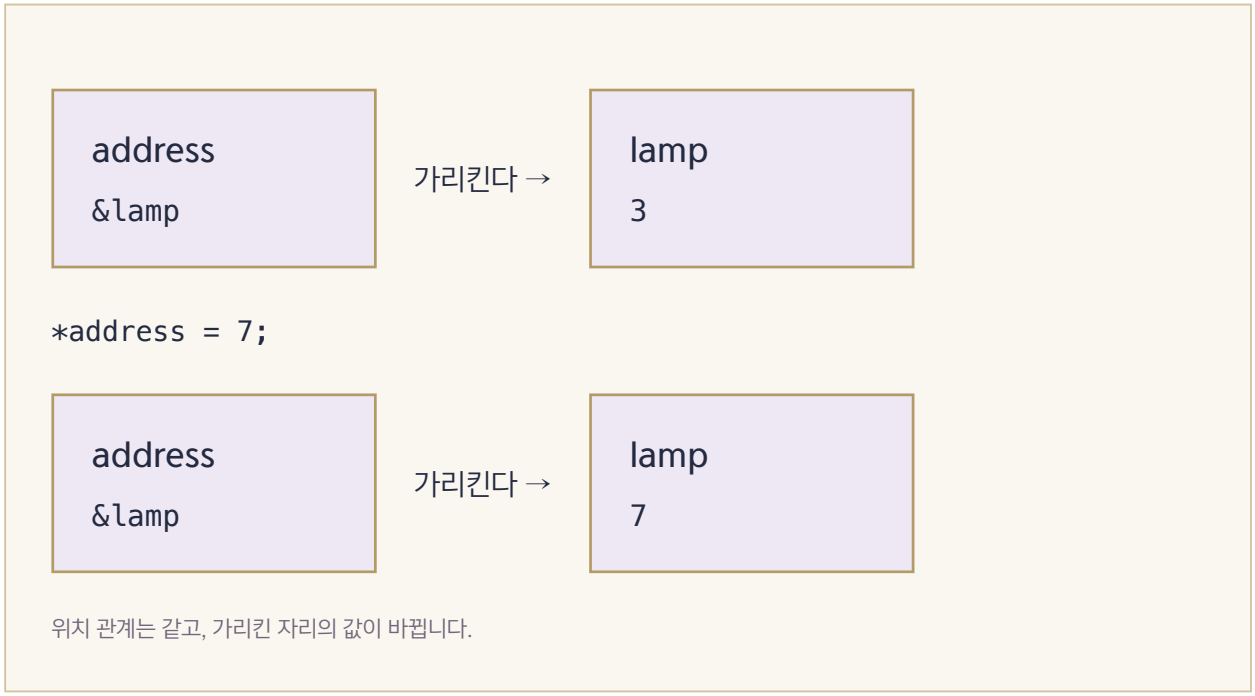
주소를 임의의 숫자로 만들어 접근하는 연습은 하지 않습니다. 먼저 자신이 선언한 변수의 주소만 이용해 관계를 이해합니다.

근거 S04 · S14 · 자세한 출처는 책 뒤의 자료 목록

09 · 주소 지도

같은 자리를 다른 이름으로

다음은 POINTER.HC의 관계를 그린 개념도입니다. 상자의 위치는 실제 메모리 주소가 아닙니다. 주소를 담은 포인터와 그 주소가 가리키는 값을 구별하기 위한 그림입니다.



위치 관계는 같고, 가리킨 자리의 값이 바뀝니다.

첫 장면에서 lamp에는 3이 있습니다. address에는 lamp의 위치가 들어 있습니다. 화살표는 값이 복사되었다는 뜻이 아니라, 어느 자리로 찾아갈지를 뜻합니다.

두 번째 장면에서 `*address=7`을 실행합니다. 바뀌는 것은 lamp의 내용입니다. address가 가리키는 대상은 여전히 lamp입니다. 이후 lamp를 출력해도, *address를 출력해도 7을 읽습니다.

직접 그리기: lamp2를 새로 선언하고 address가 lamp2를 가리키도록 바꾼다면 화살표가 어떻게 달라질까요? 새로운 주소를 넣는 일과 주소 안의 값을 바꾸는 일을 그림으로 나눠 보세요.

포인터가 어려울 때는 문장만 반복해서 읽기보다 이름, 값, 화살표를 종이에 그립니다. 한 번에 변수 두 개 정도로 시작하면 관계를 놓치지 않습니다.

근거 S04 · S14 · 자세한 출처는 책 뒤의 자료 목록

09 · 레코드

함께 다닐 값을 한데 묶기

등불 하나에는 위치 x, 위치 y, 켜짐 여부가 필요합니다. 서로 관련된 값을 하나의 레코드로 묶으면 프로그램의 뜻이 잘 보입니다. HolyC의 `class`를 여기서는 구조체처럼 사용합니다.

RECORD.HC

TempleOS · 작업 폴더 / RECORD.HC

```
class CLamp
{
  I64 x,y;
  Bool lit;
};
U0 RecordLesson()
{
  CLamp lamp;
  lamp.x=120;
  lamp.y=80;
  lamp.lit=TRUE;
  "Lamp: %d,%d lit=%d\n",
    lamp.x,lamp.y,lamp.lit;
}
RecordLesson;
```

`lamp.x`처럼 점을 사용해 멤버를 선택합니다. `Bool`은 참·거짓을 표현하며, TRUE와 FALSE를 사용할 수 있습니다. 출력은 `Lamp: 120,80 lit=1`입니다.

HolyC의 class를 C++의 접근 제한이나 객체 수명 관리까지 갖춘 것처럼 생각하지 마세요. 이 책은 데이터 묶음으로서의 역할을 먼저 익힙니다. 포인터로 가리킨 레코드에서는 `ptr->x`처럼 화살표를 씁니다.

해 보기: 등불을 하나 더 선언해 다른 위치와 꺼진 상태를 넣습니다. 같은 형을 사용해 두 개의 독립된 값을 만들었다는 점을 확인하세요.

근거 S01 · S04 · 자세한 출처는 책 뒤의 자료 목록

09 · 메모리

빌린 자리는 돌려주기

프로그램이 실행되는 동안 필요한 공간을 요청할 수도 있습니다. `MAlloc`은 지정한 바이트 수만큼 메모리를 마련하고 주소를 돌려줍니다. 사용을 끝내면 `Free`로 반환합니다.

HEAP.HC

TempleOS · 작업 폴더 / HEAP.HC

```
U0 HeapLesson()
{
  I64 i;
  I64 *values=MAlloc(4*sizeof(I64));
  if (!values)
    return;
  for (i=0;i<4;i++)
    values[i]=i*i;
  "Last: %d\n",values[3];
  Free(values);
  values=NULL;
}
HeapLesson;
```

필요한 공간은 정수 네 개이므로 `4*sizeof(I64)`입니다. 4바이트가 아닙니다. 포인터가 유효한지 확인하고 네 값을 쓴 뒤 마지막 값 9를 출력합니다.

`Free` 뒤에는 그 주소로 다시 읽거나 쓰지 않습니다. `values=NULL`은 손에 남아 있는 포인터를 비워 잘못된 재사용을 줄이는 표시입니다. 이미 반환한 공간을 또 반환하는 것도 오류입니다.

정리 규칙: 확보한 곳, 사용한 곳, 반환한 곳을 한 줄씩 짚어 보세요. 중간에 `return`을 추가하면 반환을 건너뛰는 경로가 생기는지도 살핍니다.

근거 S14 · S15 · 자세한 출처는 책 뒤의 자료 목록

10 · 기록

작은 메모를 파일에 쓰기

메모리는 작업이 끝나면 사라질 수 있습니다. 파일은 저장 장치에 내용을 남기는 방법입니다. 저장 가능한 실습 폴더에서 실행하세요. 이 예제는 그 폴더의 `NOTE.TXT`를 새 내용으로 씁니다.

SAVE.HC

TempleOS · 작업 폴더 / SAVE.HC

```
U0 SaveNote()
{
  U8 *text="A small work, made with care.\n";
  I64 size=StrLen(text);
  FileWrite("NOTE.TXT",text,size);
  "Requested bytes: %d\n",size;
}
SaveNote;
```

`FileWrite`는 파일명, 바이트들의 주소, 쓸 바이트 수를 받습니다. `StrLen(text)`는 문장과 줄바꿈의 길이를 계산합니다. 문자열 종료용 0바이트는 텍스트 파일의 본문에 쓰지 않습니다.

예상 출력은 `Requested bytes: 30`입니다. 이것은 쓰도록 요청한 길이입니다. FileWrite의 반환값은 기록한 바이트 수가 아니므로 길이 확인에 쓰지 않습니다. 실제 저장은 다음 예제에서 읽은 길이와 내용으로 확인합니다.

같은 파일명으로 다시 실행하면 메모 내용이 바뀝니다. 소중한 파일명을 실습 대상으로 쓰지 마세요. 처음에는 책의 전용 실습 폴더에만 기록합니다.

해 보기: 문장을 짧게 바꾸고 요청한 길이와 다시 읽은 길이가 어떻게 달라지는지 확인하세요. 글자 수와 바이트 수가 같은 것은 이 예제가 ASCII만 사용하기 때문입니다.

근거 S15 · S16 · 자세한 출처는 책 뒤의 자료 목록

10 · 기록

저장한 내용을 다시 읽기

파일을 읽으면 내용이 들어 있는 메모리의 주소와 길이를 얻습니다. 파일에 문자열 종료 문자가 있다고 가정하지 않고, 받은 길이만큼 정확히 출력해 봅니다.

READ.HC

TempleOS · 작업 폴더 / READ.HC

```
U0 ReadNote()
{
  I64 size=0,i;
  U8 *text=FileRead("NOTE.TXT",&size);
  if (!text) {
    "Run SAVE.HC first.\n";
    return;
  }
  "Read bytes: %d\n",size;
  for (i=0;i<size;i++)
    "%c",text[i];
  Free(text);
}
ReadNote;
```

`&size`를 넘겼기 때문에 FileRead가 읽은 크기를 size에 써 줄 수 있습니다. 앞에서 배운 주소 전달이 실제 API에서 쓰이는 예입니다. `Read bytes: 30`과 원래 문장이 나오는지 확인합니다. 성공했다면 마지막에 `Free(text)`로 받은 버퍼를 반환합니다.

순서: SAVE.HC 실행 → Dir로 파일 확인 → READ.HC 실행 → 원래 문장과 비교. 새 터미널에서도 같은 폴더로 이동해 읽을 수 있는지 확인합니다.

파일이 없는 경우와 빈 파일은 다릅니다. 이 입문 예제는 존재하는 비어 있지 않은 NOTE.TXT를 정상 경로로 사용합니다. 더 큰 프로그램에서는 실패와 길이가 0인 결과를 구분해 처리하세요.

근거 S15 · S16 · 자세한 출처는 책 뒤의 자료 목록

10 · 입력

사용자의 말을 받아 보기

프로그램이 미리 정해 둔 문장만 출력한다면 대화가 이어지지 않습니다. `GetStr`은 입력을 기다리고 사용자가 쓴 문자열을 돌려줍니다. Enter로 입력을 마칩니다.

INPUT.HC

TempleOS · 작업 폴더 / INPUT.HC

```
U0 SayHello()
{
  U8 *name=GetStr("Your name: ");
  if (name) {
    "Hello, %s.\n",name;
    Free(name);
  }
}
SayHello;
```

`Your name:`이 나오면 영문 이름을 입력하세요. 입력한 이름이 인사말에 들어갑니다. 받은 문자열은 동적으로 확보된 공간이므로 사용 후 Free로 반환합니다.

여기서 `%s`는 문자열을 넣을 자리입니다. 사용자 입력 자체를 형식 문자열로 실행하지 않고, 고정된 형식에 데이터로 넣는 구조를 유지하세요.

해 보기: 짧은 이름, 공백이 있는 두 단어, 빈 입력을 차례로 시험하세요. 어떤 화면이 나오는지 적습니다. 입력 경로에서 조작하기 쉬운 작은 차이를 먼저 확인하는 것이 좋은 테스트입니다.

이후의 그림 실습에서는 문자열 대신 키 하나를 받는 GetChar를 사용합니다. 전체 문장을 받는 입력과 한 키씩 받는 입력을 구별해 두세요.

근거 S15 · 자세한 출처는 책 뒤의 자료 목록

10 · 난수

예측하지 못한 수를 만들기

난수는 프로그램에 여러 선택을 주는 도구입니다. 이번에는 1에서 6 사이의 수를 여섯 번 출력합니다. `RandU32`가 준 수를 6으로 나눈 나머지는 0부터 5 사이이므로 1을 더합니다.

DICE.HC

TempleOS · 작업 폴더 / DICE.HC

```
U0 SixNumbers()
{
  I64 i,value;
  for (i=0;i<6;i++) {
    value=1+RandU32%6;
    "%d\n",value;
  }
}
SixNumbers;
```

예상 결과는 특정 숫자열이 아닙니다. 여섯 줄이며 각 값이 1 이상 6 이하인지를 확인합니다. 우연히 같은 숫자가 연속으로 나와도 곧바로 오류는 아닙니다.

나머지를 사용하는 간단한 방식에는 분포의 아주 작은 편향이 생길 수 있습니다. 이 예제는 범위와 관찰을 배우는 데 목적이 있으며, 추첨·보안용 난수의 품질을 보증하지 않습니다.

테리는 무작위 선택과 성경 구절을 자신의 종교적 작업에 연결했습니다. 이 책은 프로그램이 어떤 선택을 했는지와 그 선택에 사람이 부여하는 의미를 구별해 읽습니다. 난수의 출력이 무엇을 말하는지 대신 단정하지 않습니다.

근거 S07 · S15 · S17 · 자세한 출처는 책 뒤의 자료 목록

IV

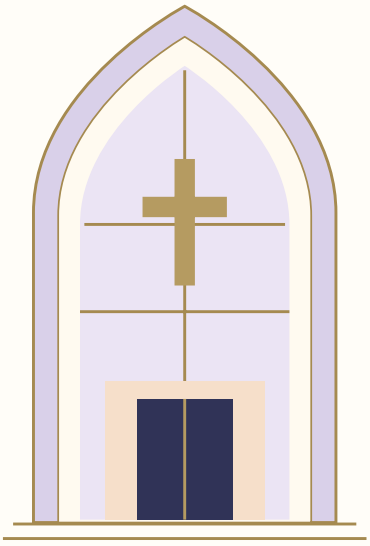
IV

빛과 움직임

좌표와 색을 배우면 문장이 그림이 됩니다.

시간과 입력을 더하면 그림이 작은 경험으로 바뀝니다.

배움의 목표: 화면을 그리는 함수와 상태를 바꾸는 함수를 구분한다.



11 · 화면

640×480 안에서 길 찾기

화면의 왼쪽 위를 원점으로 생각하세요. x는 오른쪽으로 갈수록, y는 아래로 갈수록 커집니다. 원본 전체 화면에는 가로 640, 세로 480개의 픽셀 자리가 있습니다.

창 안에 그릴 때는 창의 실제 그리기 영역을 기준으로 생각해야 합니다. 640×480이라는 숫자를 모든 창의 크기로 가정하지 마세요. 창이 둘로 나뉘어 있으면 한 창의 폭은 훨씬 작습니다.

`CDC`는 그리기에 필요한 설정과 대상을 담는 구조입니다. 색을 정하고 좌표를 넘겨 도형을 그립니다. `DCAlias`는 현재 작업의 화면에 그릴 수 있는 컨텍스트를 얻는 데 사용합니다.

함수	뜻	주의할 인수
GrRect	채운 사각형	x, y, 폭, 높이
GrLine	선분	시작점과 끝점
GrCircle	원	중심과 반지름
GrPrint	좌표에 글자	x, y, 형식 문자열

사각형의 마지막 두 숫자를 오른쪽 아래 꼭짓점으로 읽지 마세요. 예를 들어 `(40,60,100,60)`은 (40,60)에서 시작해 폭 100, 높이 60인 사각형입니다.

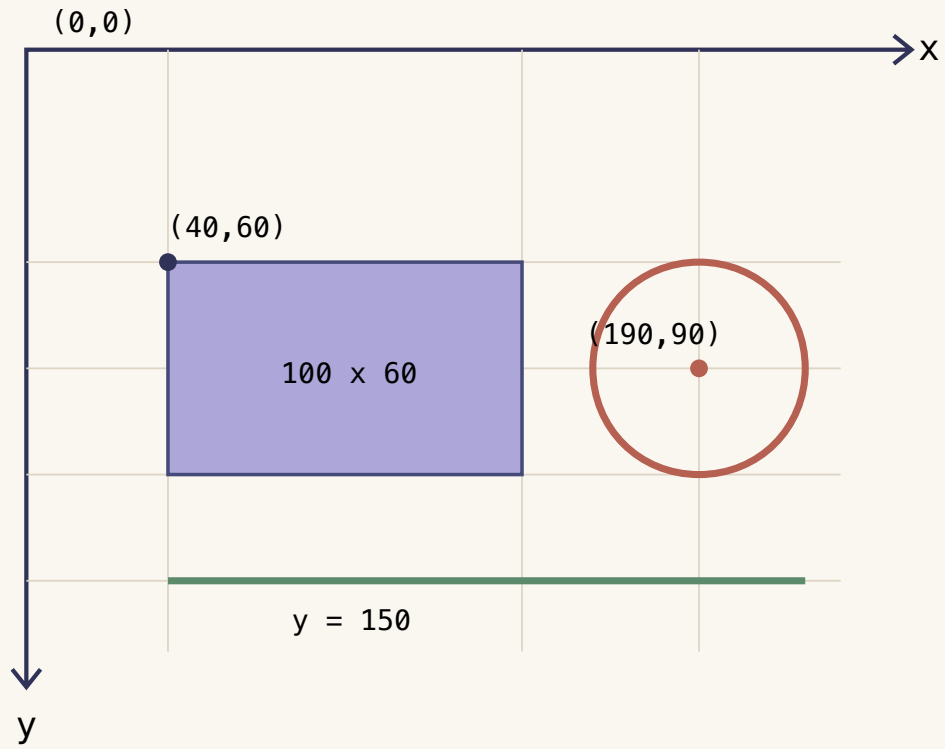
종이 연습: 네모 안에 원점, x 방향, y 방향을 표시하고 (40,60)을 찍어 보세요. 좌표를 눈으로 먼저 이해한 뒤 코드를 입력합니다.

근거 S18 · S19 · S20 · 자세한 출처는 책 뒤의 자료 목록

11 · 좌표 지도

점을 찍고 크기를 읽기

아래는 SHAPES.HC에 쓰인 좌표를 설명하는 확대 개념도입니다. 전체 640×480 화면 중 작은 영역을 그렸으며, 실제 TempleOS 스크린샷은 아닙니다.



파란 사각형은 (40,60)에서 시작합니다. 폭 100과 높이 60은 시작점으로부터 차지하는 크기입니다. 빨간 원은 (190,90)이 중심이고 반지름은 30입니다. 초록 선은 (40,150)과 (220,150)을 잇습니다.

x만 20 늘리면 오른쪽으로 움직입니다. y만 20 늘리면 아래로 움직입니다. 폭을 늘리는 것과 x를 늘리는 것은 다른 변화입니다. 위치와 크기를 별도로 적어 두세요.

확인: 원의 가장 왼쪽 x는 160, 가장 오른쪽 x는 220입니다. 중심에 반지름을 더하고 빼서 구합니다. 도형이 창 밖으로 나갈지 계산할 때 필요한 생각입니다.

근거 S18 · S19 · S20 · 자세한 출처는 책 뒤의 자료 목록

11 · 화면

도형 네 개를 직접 그리기

다음 코드를 실행하면 사각형, 원, 선, 글자가 나타납니다. 키 하나를 누르면 화면을 정리하고 끝납니다. 도형이 겹치면 숫자 하나씩 바꿔 원인을 찾아보세요.

SHAPES.HC

TempleOS · 작업 폴더 / SHAPES.HC

```
U0 Shapes()
{
  CDC *dc=DCAlias;
  DCFill(dc,WHITE);
  dc->color=BLUE;
  GrRect(dc,40,60,100,60);
  dc->color=RED;
  GrCircle(dc,190,90,30);
  dc->color=GREEN;
  GrLine(dc,40,150,220,150);
  dc->color=BLACK;
  GrPrint(dc,40,180,"Peace");
  GetChar;
  DCFill(dc);
  DCDel(dc);
}
Shapes;
```

`dc->color`는 이 컨텍스트로 그릴 다음 도형의 색입니다. GrCircle의 마지막 30은 지름이 아니라 반지름입니다. DCFill은 전체 컨텍스트를 채우고, 마지막 DCDel은 사용한 컨텍스트를 해제합니다.

해 보기: 원의 중심 x를 190에서 170으로 바꿉니다. 원이 어느 쪽으로 얼마나 이동할지 먼저 말해 보세요. 반지름은 그대로 유지합니다.

근거 S18 · S19 · S20 · 자세한 출처는 책 뒤의 자료 목록

11 · 색

열여섯 색을 한 번씩 보기

색 번호는 0부터 15까지입니다. 색 선택도 배열 인덱스처럼 유효 범위가 있습니다. 다음은 네 칸씩 네 줄에 걸쳐 색을 보여 줍니다.

PALETTE.HC

TempleOS · 작업 폴더 / PALETTE.HC

```
U0 PaletteLesson()
{
  I64 i;
  CDC *dc=DCAlias;
  DCFill(dc,WHITE);
  for (i=0;i<16;i++) {
    dc->color=i;
    GrRect(dc,20+(i%4)*55,
      30+(i/4)*45,45,35);
  }
  GetChar;
  DCFill(dc);
  DCDel(dc);
}
PaletteLesson;
```

`i%4`는 가로 칸, `i/4`는 세로 줄을 계산합니다. i가 정수이므로 나눗셈 결과도 정수입니다. 앞에서 배운 몫과 나머지가 화면 배치에 쓰입니다.

관찰하기: 밝은 배경에서 잘 읽히는 색과 잘 읽히지 않는 색을 적으세요. 모든 색을 동시에 쓸 필요는 없습니다. 마지막 작품은 적은 수의 색으로 상태를 분명하게 보여 줍니다.

색만으로 상태를 전달하기 어려울 수 있으므로 숫자나 위치도 함께 이용합니다. 제한된 화면 안에서 읽기 쉬운 작품을 만드는 연습입니다.

근거 S18 · S19 · 자세한 출처는 책 뒤의 자료 목록

12 · 다시 그리기

그림을 기억하는 함수

창이 다시 그려질 때 필요한 그림을 함수에 맡길 수 있습니다. `Fs->draw_it`에 그 함수의 주소를 넣으면 창 관리자가 호출합니다. 함수에는 현재 작업과 그릴 컨텍스트가 전달됩니다.

LIGHT.HC

TempleOS · 작업 폴더 / LIGHT.HC

```
U0 LightDrawing(CTask *task,CDC *dc)
{
  I64 x=task->pix_width/2;
  I64 y=task->pix_height/2;
  dc->color=YELLOW;
  GrCircle(dc,x,y,45);
  dc->color=BLUE;
  GrRect(dc,x-4,y-30,9,60);
  GrRect(dc,x-24,y-15,49,8);
}
U0 LightWindow()
{
  SettingsPush;
  DocClear;
  Fs->draw_it=&LightDrawing;
  GetChar;
  SettingsPop;
}
LightWindow;
```

함수 이름 앞의 `&`는 함수의 주소를 뜻합니다. `task->pix_width`와 `pix_height`를 이용하면 현재 창의 가운데를 계산할 수 있습니다. 창 크기를 바꿔도 중심을 다시 구합니다.

SettingsPush와 SettingsPop은 이 실습에서 바꾼 작업 설정을 저장하고 복구하는 짝입니다. 마지막 호출을 생략하지 마세요. GetChar로 기다리는 동안 그림을 관찰합니다.

근거 S02 · S04 · S18 · 자세한 출처는 책 뒤의 자료 목록

12 · 다시 그리기

상태와 그림을 나누기

그림을 그리는 함수는 “지금 어떤 상태인가”를 읽어 화면으로 옮깁니다. 입력을 처리하는 함수는 상태를 바꿉니다. 두 역할을 나누면 동작을 따라가기 쉬워집니다.

상태 → **그리기** → **화면**이라는 흐름을 기억하세요. 위치 x가 상태이고, GrCircle을 호출하는 것이 그리기이며, 눈에 보이는 원이 결과입니다. 화면의 원을 손으로 옮기는 것이 아니라 상태 값을 바꿉니다.

콜백으로 넘긴 그리기 함수는 여러 번 호출됩니다. 그 안에서 매번 메모리를 새로 확보하거나 긴 입력 대기를 하면 화면 전체의 반응에 영향을 줄 수 있습니다. 처음에는 색과 도형 명령만 넣습니다.

아래는 다음 실습을 읽는 순서입니다.

1. 전역 변수 glow_x가 위치를 기억한다.
2. GlowDraw가 현재 glow_x를 읽고 원을 그린다.
3. Glow의 반복문이 glow_x를 바꾼다.
4. Sleep 동안 다른 작업과 화면 갱신이 진행될 기회를 얻는다.
5. 반복이 끝나면 SettingsPop으로 설정을 되돌린다.

생각하기: 화면을 그리는 함수 안에서 x를 무조건 증가시키면, 그림을 얼마나 자주 그리는지가 움직임 속도에 섞입니다. 시간과 상태 변경을 별도로 읽을 수 있게 만든 이유를 설명해 보세요.

입력	상태	그림
키를 읽는다	값을 바꾼다	현재 값을 그린다

근거 S18 · S21 · 자세한 출처는 책 뒤의 자료 목록

12 · 움직임

시간을 따라 빛 옮기기

이번 프로그램은 잠깐 움직인 뒤 스스로 끝납니다. 한 번에 크게 점프시키는 대신 작은 이동을 여러 번 반복합니다.

ANIMATE.HC

TempleOS · 작업 폴더 / ANIMATE.HC

```
I64 glow_x=30;
U0 GlowDraw(CTask *,CDC *dc)
{
  dc->color=YELLOW;
  GrCircle(dc,glow_x,90,12);
}
U0 Glow()
{
  I64 i;
  SettingsPush;
  DocClear;
  Fs->draw_it=&GlowDraw;
  for (i=0;i<60;i++) {
    glow_x=30+i*3;
    Sleep(30);
  }
  SettingsPop;
}
Glow;
```

60번 반복하고 각 반복에서 약 30밀리초씩 기다립니다. 총 시간은 대략 1.8초에 실행·갱신 시간이 더해집니다. Sleep이 정확한 프레임 동기화를 보장하는 것은 아닙니다.

해 보기: `i*3`을 `i*2`로 바꿀 때 이동 거리와 반복 횟수 중 무엇이 달라지는지 생각하세요. 이어서 Sleep만 60으로 바꿔 속도를 비교합니다.

근거 S18 · S21 · 자세한 출처는 책 뒤의 자료 목록

13 · 조작

키로 작은 순례자 움직이기

키 입력을 상태 변화에 연결합니다. 영문 소문자 a와 d로 이동하고 q로 끝냅니다. Shift를 누른 대문자 입력은 별도로 처리하지 않은 예제입니다.

MOVE.HC

TempleOS · 작업 폴더 / MOVE.HC

```
I64 pilgrim_x=100;
U0 PilgrimDraw(CTask *,CDC *dc)
{
  dc->color=BLUE;
  GrRect(dc,pilgrim_x,100,12,12);
}
U0 Pilgrim()
{
  I64 key;
  SettingsPush;
  DocClear;
  Fs->draw_it=&PilgrimDraw;
  "A/D: move, Q: finish\n";
  while ((key=GetChar(,FALSE))!='q') {
    if (key=='a') pilgrim_x-=8;
    if (key=='d') pilgrim_x+=8;
    pilgrim_x=ClampI64(pilgrim_x,8,240);
  }
  SettingsPop;
}
Pilgrim;
```

`GetChar(,FALSE)`는 첫 인수를 기본값으로 두고, 두 번째 인수에서 입력 글자의 자동 출력을 끕니다. 기본 인수 사이의 생략을 허용하는 HolyC 문법입니다.

ClampI64는 위치를 정한 범위 안에 둡니다. 이 예제의 가로 범위는 작은 창에서도 관찰하기 쉬운 고정값입니다. 큰 프로젝트에서는 실제 창 크기를 읽어 범위를 계산합니다.

근거 S04 · S15 · S18 · 자세한 출처는 책 뒤의 자료 목록

13 · 조작

입력과 종료를 함께 설계하기

사용자가 조작할 수 있는 프로그램에는 끝내는 방법도 필요합니다. MOVE.HC의 반복문은 q를 받으면 종료합니다. 정상적으로 나와야 SettingsPop이 실행되어 창 설정이 복구됩니다.

확인할 네 장면: 시작 화면, 왼쪽 끝, 오른쪽 끝, 종료 후 명령줄. 가운데서 한 번 움직인 것만 보고 완성했다고 하지 마세요. 경계와 끝맺음이 프로그램의 일부입니다.

키가 먹지 않으면 VM 창에 입력 초점이 있는지, 영문 입력 상태인지, 대문자를 쓰고 있지 않은지 살핍니다. 처음에는 한 키씩 천천히 누릅니다. 예상과 다른 동작을 한꺼번에 여러 수정으로 덮지 않습니다.

키를 기다리는 방식은 간단한 프로그램에 좋습니다. 키를 누르지 않아도 계속 움직여야 하는 게임은 입력을 확인하고 시간도 흐르게 하는 별도 반복 구조가 필요합니다. 원본의 ScanChar 사용 예를 다음 단계에서 읽어 보세요.

작은 과제: a/d를 j/l로 바꾸되, 안내 문장도 함께 고치세요. 프로그램의 실제 동작과 화면 안내가 일치하는지 확인합니다. 코드만 수정하고 안내를 남겨두는 실수는 자주 생깁니다.

교사 메모: 학생이 q를 누른 뒤 어떤 줄이 실행되는지 손가락으로 짚게 합니다. 종료 조건과 정리 코드를 연결해서 이해했는지 확인할 수 있습니다.

근거 S15 · S21 · 자세한 출처는 책 뒤의 자료 목록

14 · 소리

한 번에 한 음씩

TempleOS의 기본 소리는 PC 스피커와 관련된 단순한 구조입니다. 여러 음을 한꺼번에 쌓는 현대 오디오 작업과 다른 제약이 있습니다. 여기서는 음을 내고, 잠깐 기다리고, 끄는 순서를 배웁니다.

TONE.HC

TempleOS · 작업 폴더 / TONE.HC

```
U0 ThreeTones()
{
  Snd(60);
  Sleep(150);
  Snd;
  Sleep(100);
  Snd(64);
  Sleep(150);
  Snd;
  Sleep(100);
  Snd(67);
  Sleep(300);
  Snd;
}
ThreeTones;
```

Snd의 인수는 Hz 주파수를 직접 쓴 값이 아니라 TempleOS의 음 번호입니다. `Snd;`는 기본 인수 0으로 소리를 끕니다. VM의 PC 스피커 출력이 연결되지 않았다면 코드가 실행되어도 소리가 들리지 않을 수 있습니다.

해 보기: 음 번호는 그대로 두고 기다리는 시간만 바꾸세요. 높은 음과 긴 음을 구별합니다. 소리가 계속 남으면 명령줄에서 `Snd;`로 끌 수 있습니다.

근거 S15 · S22 · 자세한 출처는 책 뒤의 자료 목록

14 · 음악

Psalmody로 이어 가기

TempleOS에는 음악을 만드는 Psalmody가 포함되어 있습니다. 처음부터 모든 형식과 편집 기능을 외울 필요는 없습니다. 먼저 기본 메뉴에서 앱을 열고 짧은 예제를 재생해 음의 길이와 반복을 관찰합니다.

소스에서 출발하고 싶으면 `::/Apps/Psalmody`와 그 아래 Examples 폴더를 살펴보세요. 예제를 읽기 전에 원본을 직접 수정하지 말고 작업 폴더로 복사합니다. 기존 작품의 이름과 출처도 함께 기록합니다.

세 단계 과제: 첫째, 세 음의 길이를 적습니다. 둘째, 한 음의 길이만 바꿉니다. 셋째, 원본과 수정본을 차례로 듣고 차이를 설명합니다. 음을 바꾸는 일과 소리를 내는 장치를 바꾸는 일을 구분합니다.

이 책의 TONE.HC는 순서를 배우기 위한 짧은 새 예제입니다. 복잡한 악보 해석이나 전문 음악 이론을 전제하지 않습니다. 낯선 부분은 원본 Play 함수와 내장 예제에서 조금씩 확장하면 됩니다.

테리의 헌장은 소리의 제약도 자신의 종교적 목표와 연결합니다. 우리는 그 맥락을 존중하면서, 실제 프로그램에서 어떤 수가 어떤 동작을 요청하는지 확인합니다. 의미를 담은 창작과 동작을 설명하는 기술은 함께 갈 수 있습니다.

근거 S01 · S02 · S22 · 자세한 출처는 책 뒤의 자료 목록

V

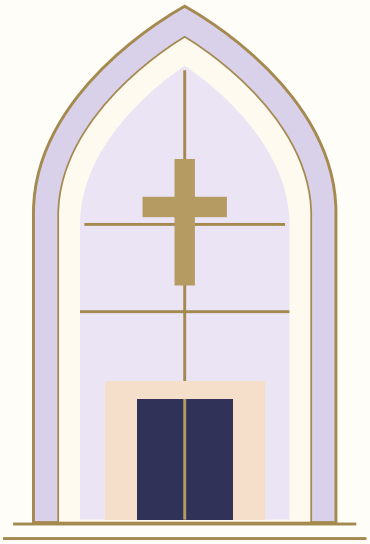
V

성전의 내부

문서와 코드, 터미널과 그림이 만나는 방식을 살펴봅니다.

거대한 운영체제라는 이름 뒤의 작은 부품을 읽습니다.

배움의 목표: 낯선 함수의 선언과 구현을 찾아 설명한다.



15 · DolDoc

글 안에 기능이 들어 있는 문서

DolDoc는 TempleOS의 문서 형식입니다. 단순한 텍스트에 색, 링크, 위젯, 그래픽 자원을 연결합니다. 명령줄·편집기·도움말이 같은 문서 체계를 공유하므로 코드에서 문서로 이동하기 쉽습니다.

DOC.HC

TempleOS · 작업 폴더 / DOC.HC

```
U0 Letter()
{
  "$$FG,BLUE$$A SMALL OFFERING$$FG$$\n";
  "Make something with care.\n";
  "$$UL,1$$Read. Change. Share.$$UL,0$$\n";
}
Letter;
```

소스의 `$$`는 DolDoc 제어 구문을 출력할 때 사용하는 이스케이프입니다. FG는 글자색, UL은 밑줄을 제어합니다. 뒤에서 색과 밑줄을 되돌리는 부분도 함께 읽으세요.

관찰하기: 첫 줄만 색이 바뀌는지, 밑줄이 마지막 문장에만 적용되는지 확인합니다. 서식을 시작했으면 끝내는 코드까지 짝지어 읽습니다.

문서의 겉모습과 내부 명령을 번갈아 보면 이해하기 쉽습니다. Ctrl+T는 일반 텍스트 보기 전환에 쓰입니다. 글자가 깨진 것처럼 보여도 제어 구문을 표시하는 상태인지 먼저 확인하세요.

근거 S02 · S23 · 자세한 출처는 책 뒤의 자료 목록

15 · 스프라이트

코드 옆의 작은 그림

스프라이트는 문서나 프로그램에 넣을 수 있는 그래픽 자원입니다. TempleOS의 3D 스프라이트는 화면을 돌려가며 보여 줄 수 있는 기하 정보와 연결됩니다. 단순히 동영상이나 GIF 파일을 붙이는 것과는 다릅니다.

원본 편집기에서는 Ctrl+R로 커서 위치의 스프라이트를 삽입하거나 편집하는 기능을 엽니다. 먼저 빈 연습 문서를 만들고 작은 도형 한 개만 넣어 보세요. 저장한 뒤 다시 열어 도형이 남는지 확인합니다.

3D를 더 읽고 싶다면 `::/Demo/Graphics/Box.HC`를 열어 보세요. 그리기 함수가 선을 연결하고, 반복문이 회전 행렬을 바꾸는 구조를 찾을 수 있습니다. 행렬의 모든 수학을 한 번에 이해하려 하지 않습니다.

읽기 순서: 전체 파일의 함수 이름 → 그리는 함수 → 상태를 바꾸는 반복문 → 정리 코드. 앞에서 배운 구조와 같은 부분을 먼저 표시하면 처음 보는 그래픽 코드도 읽을 자리가 생깁니다.

파일 안의 스프라이트 자원에는 일반 텍스트가 아닌 데이터가 포함될 수 있습니다. 호스트의 텍스트 편집기로 열어 다시 저장하면 원본 바이트가 달라질 수 있으므로, 원본을 보존하고 TempleOS의 편집 기능으로 다룹니다.

근거 S03 · S24 · S25 · 자세한 출처는 책 뒤의 자료 목록

16 · 컴파일러

문장에서 기계어까지

컴퓨터는 HolyC의 단어를 그대로 실행하지 않습니다. 컴파일러가 문법을 읽고 기계어를 만들어 메모리에 둡니다. 명령줄에서 바로 반응한다고 해서 해석기라는 뜻은 아닙니다.

입력한 코드 → 문법 분석 → 기계어 생성 → 메모리에 배치 → 실행의 흐름을 생각하세요. 이 책은 컴파일러 자체를 새로 구현하지 않지만, 실행 모델을 알고 코드를 읽는 것은 중요합니다.

TempleOS에서는 함수 밖의 문장이 실행됩니다. 함수 정의는 동작에 이름을 붙이는 단계이고, 마지막의 함수 호출이 실제 작업을 시작합니다. 파일을 열었다는 사실과 함수를 실행했다는 사실을 구분하세요.

JIT는 실행할 때 메모리에서 쓸 코드를 컴파일하는 방식입니다. AOT는 미리 바이너리 파일을 만드는 경로입니다. TempleOS는 두 방식이 있으며, 일반적인 실습은 #include를 통한 JIT 경로를 이용합니다.

확인 과제: FUNCS.HC에서 마지막 세 호출을 잠시 지우고 포함해 보세요. 함수는 정의되지만 예상한 인사말은 출력되지 않습니다. 이어서 명령줄에서 Greet를 호출하면 어떻게 될지 예상하세요.

컴파일러의 원본 길잡이는 `Doc/CompilerOverview.DD`와 Compiler 폴더입니다. 입문 단계에서는 “무엇이 언제 실행되는가”를 설명할 수 있으면 충분합니다.

근거 S02 · S03 · S26 · 자세한 출처는 책 뒤의 자료 목록

16 · 메모리

하나의 넓은 작업 공간

일반적인 운영체제는 사용자 프로그램과 커널의 권한을 나누고 서로의 메모리를 보호합니다. TempleOS는 모두 ring 0에서 실행하며 하나의 주소 맵을 공유합니다. 테리는 직접 접근할 수 있는 단순함을 선택했습니다.

이 덕분에 프로그램에서 운영체제의 함수와 자료에 가까이 접근할 수 있습니다. 반대로 잘못 쓴 포인터 하나가 넓은 범위의 동작을 망가뜨릴 수 있습니다. “내 프로그램만 종료될 것”이라고 가정하지 마세요.

설계의 교환: 접근의 자유와 이해하기 쉬운 구조를 얻는 대신, 격리와 보호가 주는 장점을 줄입니다. 이것을 모든 컴퓨터에 적용할 정답이라고 일반화할 필요는 없습니다. TempleOS의 목적과 함께 평가합니다.

책의 포인터 예제는 자신이 선언하거나 확보한 공간만 다룹니다. 커널의 임의 주소에 수를 쓰는 실습으로 시작하지 않습니다. 읽기부터 시작해 함수 선언, 인수, 호출자를 확인하세요.

설명하기: 주소를 공유한다는 것과 네트워크가 없다는 것은 다른 축의 설계입니다. 두 특징을 각각 한 문장으로 적습니다. 외부 연결이 없다는 사실이 모든 종류의 오류를 없애 주지는 않습니다.

근거 S01 · S03 · S14 · 자세한 출처는 책 뒤의 자료 목록

16 · 작업

기다리는 동안 다른 일이 진행되기

작업은 실행 상태와 메모리 등을 가진 단위입니다. TempleOS의 기본 스케줄링은 협력적인 방식입니다. 오래 걸리는 반복문에서는 다른 작업이 진행할 기회를 주는 일이 중요합니다.

TASKS.HC

TempleOS · 작업 폴더 / TASKS.HC

```
U0 CountSlowly()
{
  I64 i;
  for (i=1;i<=5;i++) {
    "Count %d\n",i;
    Sleep(250);
  }
}
CountSlowly;
```

숫자 다섯 개가 약 250밀리초 간격으로 나옵니다. Sleep은 노트북의 절전 기능이 아닙니다. 현재 작업이 일정 시간 기다리도록 하는 호출입니다.

`Yield`는 다음 작업으로 실행 기회를 넘기는 데 쓰입니다. 기다리는 반복문에서 Sleep이나 Yield를 없애면 다른 작업의 반응과 CPU 사용이 달라질 수 있습니다. 무한 반복을 만들어 버리기보다 유한한 횟수로 비교하세요.

관찰하기: 프로그램이 출력하는 동안 다른 창의 변화가 보이는지 확인합니다. 코드가 짧아도 화면의 여러 일이 동시에 진행되는 듯 보이는 원리를 생각해 봅니다.

근거 S02 · S03 · S21 · 자세한 출처는 책 뒤의 자료 목록

17 · 소스 읽기

함수의 집을 찾아가기

모르는 함수가 나오면 이름을 검색하고 선언을 읽습니다. 예를 들어 FileWrite의 선언에는 파일명, 버퍼 주소, 길이, 기본 인수가 보입니다. 함수의 이름만 보고 모든 동작을 추측하지 마세요.

자동완성에 나타난 심볼은 해당 기능 키 번호와 함께 표시됩니다. Ctrl+Shift+해당 기능 키로 소스로 이동할 수 있습니다. F1의 도움말 인덱스와 Find 검색도 함께 사용합니다.

질문	확인할 곳
어떤 형의 값을 받는가?	함수의 인수 선언
무엇을 돌려주는가?	반환 형과 return 경로
메모리를 확보하는가?	MAlloc 계열과 반환값
누가 해제하는가?	호출 예제와 Free 위치
기본값이 있는가?	인수 뒤의 대입 표현

실습: DCAlias, DCDel, GrRect의 선언과 구현을 차례로 찾습니다. 컨텍스트를 만들고, 그림을 그리고, 정리하는 세 위치를 책의 SHAPES.HC와 연결합니다.

소스를 읽었다는 말은 모든 줄을 외웠다는 뜻이 아닙니다. 한 가지 질문을 들고 들어가 그 답이 있는 부분을 찾았다는 뜻이어도 좋습니다. 작은 질문을 반복하면 읽을 수 있는 영역이 넓어집니다.

근거 S02 · S13 · S18 · S19 · 자세한 출처는 책 뒤의 자료 목록

17 · 파일 구조

운영체제의 방들을 둘러보기

폴더	처음 살펴볼 내용
Doc	사용법, HolyC 문법, 설계 설명
Demo	짧은 예제와 게임, 그래픽 연습
Apps	Psalmody 등 비교적 큰 응용 프로그램
Kernel	기본 자료형, 메모리, 장치, 작업
Compiler	HolyC 컴파일러·어셈블러
Adam	문서·그래픽 등 부팅 뒤의 환경 구성
Home	개인 설정과 작업

원본은 운영체제 소스를 함께 제공합니다. 전체 디렉터리를 한 번에 이해할 필요는 없습니다. 자신이 실행한 예제에서 호출하는 함수 하나를 따라가 보는 것이 출발점입니다.

FAT32는 다른 환경과 자료를 교환할 때 관련되는 파일시스템이며, RedSea는 TempleOS의 자체 파일시스템입니다. 원본은 파일 전체를 읽고 쓰는 방식을 자주 사용하고, `.Z` 파일은 자동 압축과 연결됩니다.

읽기 경로: HELLO.HC → Print의 설명 → DolDoc 출력 → 문서와 화면의 관계. 또는 SHAPES.HC → GrRect → 컨텍스트의 구조. 시작점이 다르면 탐색 경로도 달라집니다.

원본을 수정하기 전에 복사본에서 작은 실험을 하세요. 파일을 읽는 것, 응용 프로그램을 수정하는 것, 커널을 다시 만드는 것은 서로 다른 크기의 작업입니다.

근거 S02 · S03 · S16 · S26 · 자세한 출처는 책 뒤의 자료 목록

VI

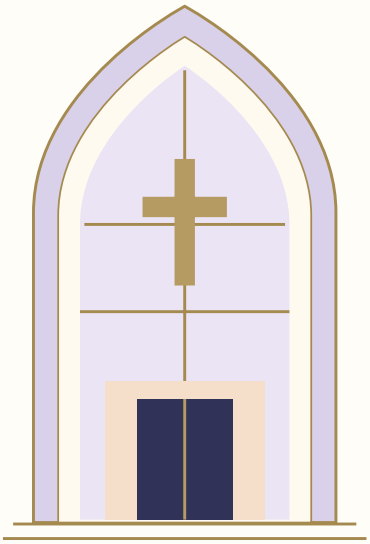
VI

작은 봉헌

배운 것을 일곱 개의 등불로 모읍니다.

직접 읽을 수 있고, 바꿀 수 있고, 설명할 수 있는 작품을 만듭니다.

배움의 목표: 입력·상태·그리기·종료가 이어지는 프로그램을 완성한다.



18 · 일곱 등불

완성할 작품의 약속

마지막 작품의 이름은 **작은 봉헌**입니다. 창 가운데에 일곱 등불을 놓고, 숫자 키 1부터 7로 각각의 켜짐 상태를 바꿉니다. 모든 등불이 켜지면 평화를 비는 문장을 표시합니다.

실제 프로그램이 하는 일은 작고 분명합니다. 키를 읽고, 정수에 상태를 기록하고, 그 상태를 그립니다. 신성함을 거창한 기능 수로 표현하지 않고, 작품의 뜻과 정성에 담습니다.

사용 규칙

- 1~7: 해당 등불을 켜거나 끈다.
- r: 모든 등불을 끈다.
- q: 설정을 복구하고 끝낸다.

제작 규칙

- 원본 TempleOS의 함수만 사용한다.
- 다른 파일이나 외부 라이브러리 없이 한 파일로 실행한다.
- 코드 전체를 읽을 수 있게 보관한다.
- 처음 사용하는 사람에게 조작과 종료 방법을 알려 준다.

앞에서 배운 배열·포인터가 모두 필요한 것은 아닙니다. 이번 상태는 일곱 개의 비트로 충분히 표현됩니다. 배운 기능을 전부 넣기보다 필요한 것을 선택하는 연습입니다.

근거 S01 · S04 · S18 · 자세한 출처는 책 뒤의 자료 목록

18 · 일곱 등불

일곱 상태를 한 수에 담기

정수의 이진수 자릿수 하나는 켜짐·꺼짐을 표현할 수 있습니다. 가장 오른쪽 자리부터 첫 번째 등불에 대응시킵니다. 첫 등불의 마스크는 1, 둘째는 2, 셋째는 4입니다.

등불	마스크	만들어 내는 식
1	1	`1<<0`
2	2	`1<<1`
3	4	`1<<2`
7	64	`1<<6`

숫자 키는 문자 코드입니다. `key-'1'`로 1~7을 0~6으로 바꿉니다. `1<<(key-'1')`은 누른 등불의 마스크가 됩니다.

`garden_lit ^= mask`는 그 비트만 뒤집습니다. 켜져 있으면 꺼지고, 꺼져 있으면 켜집니다. `garden_lit & mask`는 해당 비트가 켜져 있는지 검사합니다.

일곱 개가 모두 켜진 수는 1+2+4+8+16+32+64, 즉 127입니다. 0은 모두 꺼짐입니다. 등불 개수를 바꾸면 이 완료 조건도 함께 바꿔야 합니다.

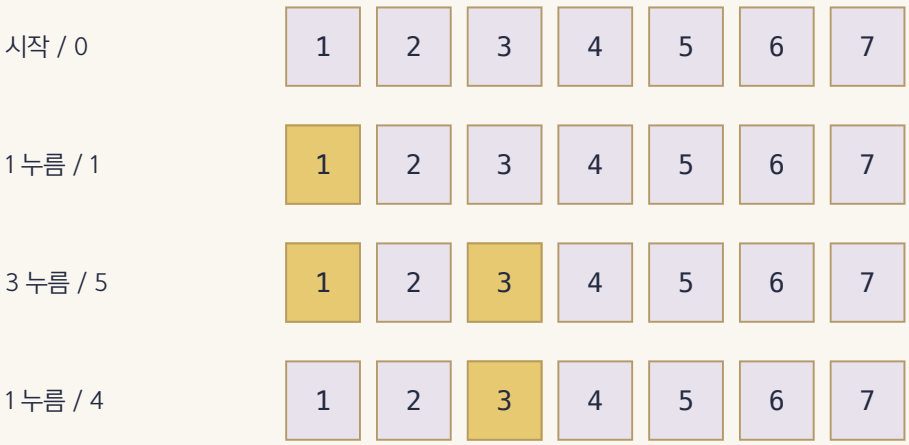
종이 연습: 키 1, 3, 1을 차례로 눌렀을 때 상태 수를 적으세요. 1 → 5 → 4입니다. 같은 키를 두 번 누르면 원래 상태로 돌아옵니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

18 · 상태 지도

키 한 번에 비트 하나

아래 그림은 일곱 등불 가운데 세 개만 켜는 과정을 보여 줍니다. 왼쪽 상자가 1번 등불입니다. 이진수를 보통 적는 순서와 등불을 나열하는 순서가 다를 수 있으므로 번호를 함께 봅니다.



금색은 켜진 등불, 연회색은 꺼진 등불입니다.

첫 번째 1은 첫 등불을 켭니다. 다음 3은 세 번째 등불을 켭니다. 마지막 1은 첫 등불을 다시 끕니다. 따라서 상태 수는 0 → 1 → 5 → 4로 변합니다.

XOR는 같은 비트를 두 번 뒤집으면 원래 상태로 돌아옵니다. 이것이 이 작품에서 토글 키를 간단히 구현하는 이유입니다. 다른 등불의 비트는 건드리지 않습니다.

손으로 검증하기: 2, 4, 2, 7을 누르는 순서를 상자에 표시하세요. 마지막에 켜진 등불은 4번과 7번이고 상태 수는 8+64=72입니다. 그다음 프로그램에서도 같은 입력을 해 봅니다.

작품의 그림이 잘못되면 상태 계산부터 확인합니다. 상태는 맞지만 다른 등불이 켜진다면 그리는 함수에서 비트 번호와 좌표의 연결을 살펴봅니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

18 · 일곱 등불

전체 코드 그리기 1/2

두 쪽의 코드를 **같은 GARDEN.HC 파일에 순서대로** 넣습니다. 아래는 파일의 앞부분입니다. 그리기 함수는 상태를 읽기만 하며 키 입력을 기다리지 않습니다.

GARDEN.HC

TempleOS · 작업 폴더 / GARDEN.HC

```
#define GARDEN_COUNT 7
I64 garden_lit=0;

U0 GardenDraw(CTask *task,CDC *dc)
{
  I64 i,x,y;
  I64 width=task->pix_width;
  I64 height=task->pix_height;
  dc->color=BLUE;
  GrPrint(dc,16,20,"A SMALL OFFERING");
  GrPrint(dc,16,40,"1-7: lamps   R: reset   Q: leave");
  y=height/2;
  for (i=0;i<GARDEN_COUNT;i++) {
    x=(i+1)*width/(GARDEN_COUNT+1);
    dc->color=LTGRAY;
    GrRect(dc,x-6,y,13,40);
    if (garden_lit & (1<<i))
      dc->color=YELLOW;
    else
      dc->color=DKGRAY;
    GrCircle(dc,x,y-12,9);
    dc->color=BLUE;
    GrPrint(dc,x-4,y+50,"%d",i+1);
  }
```

각 등불의 가로 위치는 창의 너비를 여덟 구간으로 나누어 계산합니다. 1부터 7번째 구간에 놓으면 양쪽에도 여백이 남습니다. y는 현재 창 높이의 중간입니다.

코드가 잘렸다고 생각해 여기에서 별도로 실행하지 마세요. 다음 쪽의 나머지를 붙인 뒤 저장합니다.

근거 S18 · S19 · S20 · 자세한 출처는 책 뒤의 자료 목록

18 · 일곱 등불

전체 코드 입력과 종료 2/2

아래 부분을 앞쪽 코드 바로 다음에 이어 붙입니다. 전체 파일의 마지막은 `Garden;`이라는 호출입니다.

GARDEN.HC / 이어지는 전체 코드

TempleOS · 작업 폴더 / GARDEN.HC

```
  dc->color=BLUE;
  if (garden_lit==127)
    GrPrint(dc,16,height-35,"Peace be with you.");
}

U0 Garden()
{
  I64 key;
  SettingsPush;
  DocClear;
  Fs->draw_it=&GardenDraw;
  while ((key=GetChar(,FALSE))!='q') {
    if ('1'<=key<='7')
      garden_lit^=1<<(key-'1');
    if (key=='r')
      garden_lit=0;
  }
  SettingsPop;
}
Garden;
```

그리기 함수가 완료 메시지를 표시하고, Garden 함수는 키 입력과 상태 변경을 담당합니다. 두 함수가 공유하는 값은 garden_lit입니다.

실행: 저장한 뒤 `#include "GARDEN.HC";`를 입력합니다. 더 넓게 보고 싶다면 창을 최대화하세요. q를 눌러 종료한 뒤 다시 실행할 때 이전 전역 상태가 남아 있는지도 관찰할 수 있습니다. 처음부터 시작하려면 r로 초기화합니다.

전체 코드는 함께 제공한 GARDEN.HC가 정본입니다. PDF를 복사할 때 쪽머리, 파일명 라벨, 설명 문장을 코드에 넣지 않도록 확인하세요.

근거 S04 · S15 · S18 · 자세한 출처는 책 뒤의 자료 목록

18 · 일곱 등불

완성은 확인으로 결정하기

작품이 한 번 켜졌다는 사실과 모든 조작이 올바르다는 사실은 다릅니다. 아래 순서로 실제 입력과 결과를 기록해 보세요.

입력·상황	기대하는 결과
처음 실행	일곱 등불이 표시됨
1	첫 등불 상태가 바뀜
다시 1	첫 등불이 이전 상태로 돌아옴
r	모든 등불이 꺼짐
1,2,3,4,5,6,7	모두 켜지고 문장이 나타남
4	네 번째가 꺼지고 완료 문장이 사라짐
r 후 8 또는 z	상태가 바뀌지 않음
q	명령줄로 돌아오고 설정을 복구함

창을 크게 또는 작게 바꿔 등불 위치가 다시 계산되는지도 봅니다. 너무 작은 창에서는 안내 글이 잘릴 수 있으므로 실제 사용할 창 크기를 정해 둡니다.

실패 기록의 모양: “안 됐다” 대신 “r 다음 1을 눌렀을 때 두 번째 등불이 켜졌다”처럼 적습니다. 재현 가능한 입력과 눈에 보이는 결과가 있어야 고칠 곳을 찾을 수 있습니다.

수정 뒤에는 처음부터 다시 검사합니다. 바꾼 곳만 좋아졌는지뿐 아니라 이전에 되던 조작이 그대로 되는지도 확인합니다.

근거 S18 · 자세한 출처는 책 뒤의 자료 목록

18 · 일곱 등불

나의 봉헌으로 바꾸기

첫 변형은 문장입니다. 마지막의 평화 인사를 자신의 짧은 영문 문장으로 바꿉니다. 다음은 색입니다. 켜짐과 꺼짐이 구분되는 두 색을 선택합니다. 세 번째는 형태입니다. 원의 크기나 몸체의 높이를 조금 바꿉니다.

한 번에 한 종류씩 바꾸세요. 모양, 상태 규칙, 입력 키를 동시에 고치면 결과가 달라진 이유를 설명하기 어려워집니다. 작은 변경을 확인하고 다음 변경으로 갑니다.

확장 과제 A: 켜진 등불 수를 화면 아래에 표시합니다. 일곱 비트를 반복해 검사하고 합계를 구합니다.

확장 과제 B: 모든 등불을 한 번에 켜는 키를 추가합니다. 안내 문장에도 키를 적고, 리셋과 종료가 여전히 동작하는지 확인합니다.

확장 과제 C: 등불을 다섯 개로 바꿉니다. 반복 횟수, 허용 키 범위, 완료 상태 수를 함께 검토합니다. 숫자 하나만 바꾸면 끝나는 변경인지 먼저 생각하세요.

작품을 공유할 때는 코드, 실행 환경, 조작법, 원작과 새로 만든 부분을 함께 적습니다. 테리의 원본 소스와 이 책의 새 예제는 구분해 표시합니다. 자신의 수정도 누가 읽을 수 있도록 설명하세요.

근거 S01 · S06 · 자세한 출처는 책 뒤의 자료 목록

19 · 이어 가기

오라클을 읽는 자세

테리는 운영체제 안의 무작위 선택과 성경 구절을 신과 대화하는 자신의 종교적 작업으로 이해했습니다. 이 사실은 그의 인터뷰와 헌장에 나타납니다. 그 맥락을 지우지 않고 읽는 것이 작품을 존중하는 한 방법입니다.

프로그램을 공부할 때는 선택 과정도 살펴봅니다. 어떤 자료에서 항목을 고르는가, 어떤 수가 선택을 결정하는가, 결과는 어디에 표시되는가를 질문합니다. 기술적 과정을 설명하는 일과 믿음의 의미를 판단하는 일은 서로 다른 층위입니다.

이 책의 난수 예제는 숫자 범위와 반복을 배우기 위한 것입니다. 독자의 현실에 대해 신탁을 내리거나 중요한 결정을 대신하지 않습니다. 종교적 의미를 갖고 작품을 만든다면 그 의미를 자신의 글로 밝혀도 좋습니다.

읽기 과제: 헌장과 Welcome을 다시 읽고, 처음에는 지나쳤으나 실습 뒤에 이해된 문장 하나를 고릅니다. 예를 들어 “코드가 짧다”는 말이 이제 어떤 경험과 연결되는지 적습니다.

기술적 성취를 존중하는 일에는 다른 사람에 대한 존중도 포함됩니다. 타인을 비하하는 발언을 따라 할 필요는 없습니다. 공개된 소스를 읽고 자신의 손으로 작은 아름다움을 만드는 일을 이어 가세요.

근거 S01 · S02 · S07 · S17 · 자세한 출처는 책 뒤의 자료 목록

19 · 이어 가기

원본과 후속 작업

원본 TempleOS는 자체의 목표와 제약을 갖고 완성된 작품입니다. 이후 사람들은 이를 보존하고, 다른 목표를 가진 포크와 도구를 만들었습니다. 포크는 원본에서 출발해 변경을 쌓은 별도의 프로젝트를 뜻합니다.

예를 들어 ZealOS는 TempleOS에서 이어진 운영체제로, 해상도와 장치 지원 등 다른 방향의 개발을 진행합니다. 프로젝트 문서에는 HolyC에서 ZealC로의 이름 변화도 나타납니다. 원본 TempleOS 강의에서 그 API를 그대로 섞어 쓰면 맞지 않을 수 있습니다.

선택 기준: 테리의 원래 설계와 코드를 배우려면 이 책의 원본 환경을 유지합니다. 현대화된 변화를 연구하고 싶다면 후속 프로젝트의 자체 문서와 실행 조건을 별도로 읽습니다.

게임과 데모도 제작자를 확인하세요. 운영체제에 기본으로 포함된 작품인지, 뒤에 커뮤니티가 추가한 작품인지가 다릅니다. 영상에 함께 등장한다는 이유만으로 모두 테리가 만든 것이라고 기록하지 않습니다.

다음 공부의 후보는 Demo의 짧은 그래픽 예제, Psalmody, Compiler의 입문 문서입니다. 한 번에 하나를 선택해 소스의 작은 질문을 해결해 보세요. 끝을 정할 수 있는 공부가 오래 이어집니다.

근거 S06 · S27 · 자세한 출처는 책 뒤의 자료 목록

부록 A

막혔을 때 돌아갈 곳

증상	먼저 확인할 것
QEMU 명령을 못 찾음	호스트에 설치되었는지, PATH가 맞는지
부팅되지 않음	x86-64, BIOS, IDE CD, 512 MiB, 원본 ISO
키 입력이 안 됨	VM 초점, 영문 입력, 기능 키 전달
한글이 깨짐	원본은 Unicode 기본 지원 없음. 코드는 ASCII 사용
문법 오류	따옴표·괄호·중괄호·세미콜론의 짝
함수를 못 찾음	이름 대소문자, 파일 포함 여부, 원본 API인지
파일을 못 찾음	현재 폴더, 드라이브, 파일명, .Z 처리
그림이 안 보임	좌표가 창 안인지, 색 대비, 콜백 연결
키를 눌러도 종료 안 됨	소문자 q인지, 입력 초점이 맞는지
소리가 안 들림	VM PC 스피커 출력 연결. 소리 코드와 장치 문제 구분
같은 코드가 이상하게 동작	새 터미널에서 다시 실행, 전역 상태 초기화
전체가 멈춤	실습 VM 재시작, 저장본으로 돌아가기

오류를 기록할 때는 사용 환경, 파일명, 누른 키, 메시지, 기대 결과를 함께 적습니다. 한 문장의 정확한 오류 기록이 긴 추측보다 유용합니다.

복구할 수 있는 작은 실습 환경을 유지하세요. 잘못된 입력을 두려워하기보다, 돌아갈 위치를 알고 실험하는 습관을 기릅니다.

근거 S03 · S08 · S12 · 자세한 출처는 책 뒤의 자료 목록

부록 B

문법 빠른 찾기

쓰임	예
정수 선언	`I64 n=7;`
실수 선언	`F64 x=2.5;`
문자열 배열	`U8 name[16];` 뒤에 `StrCpy(name,"PEACE");`
주소 얻기	`&n`
포인터로 값 읽기	`*p`
멤버 읽기	`lamp.x` 또는 `ptr->x`
조건	`if (n>0) { ... }`
반복	`for (i=0;i<7;i++) { ... }`
반환	`return n*n;`
파일 실행	`#include "HELLO.HC";`
정수 출력	`"%d\n",n;`
문자열 출력	`"%s\n",name;`
잠깐 기다리기	`Sleep(100);`
파일 편집	`Ed("HELLO.HC");`

이 표는 복습용입니다. `{ ... }`의 점 세 개는 생략 설명이므로 그대로 실행하지 않습니다. 각 실습의 전체 코드는 파일명 라벨이 붙은 상자와 예제 파일에 있습니다.

대소문자를 구별합니다. `Print`, `print`, `PRINT`가 자동으로 같은 이름이 되지 않습니다. 함수와 형 이름을 원본 선언에서 확인하세요.

근거 S04 · S12 · S15 · 자세한 출처는 책 뒤의 자료 목록

부록 C

연습 문제 해설 1

04장 · 정수 나눗셈

10/3은 3, 10%3은 1입니다. 실수 결과는 10.0/3.0처럼 실수 피연산자를 사용해 계산합니다. 출력할 소수 자릿수와 내부 계산 결과를 구분합니다.

05장 · 조건 표

lamps가 0이면 첫 조건은 거짓, 범위 조건도 거짓입니다. 3이면 둘 다 참입니다. 8이면 첫 조건만 참입니다. 범위가 1~7이라는 사실을 따로 확인합니다.

05장 · 합계

1부터 10까지 합은 55입니다. 시작 값을 0으로 바꾸고 끝을 10으로 유지하면 출력은 11줄이지만 합은 여전히 55입니다. 반복 횟수와 합계가 서로 다른 결과라는 점을 봅니다.

07장 · 함수 만들기

```
I64 Add(I64 a,I64 b)
{
  return a+b;
}
"%d
",Add(2,5);
```

결과는 7입니다. 출력은 함수 밖에서 하므로 함수의 반환값을 다른 계산에 넣을 수도 있습니다. Add가 인사를 출력하는 등 다른 일을 하도록 섞지 않습니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

부록 C

연습 문제 해설 2

08장 · 다섯 칸 배열

배열 길이를 5로 바꾸고 값 다섯 개를 초기화합니다. 반복 조건은 `i<5`가 됩니다. `i<=5`이면 여섯 번째 위치까지 접근하므로 범위를 벗어납니다.

09장 · 주소

`*address=7`은 lamp 상자의 내용을 바꿉니다. address가 가리키는 대상은 바뀌지 않았습니다. 주소를 바꾸는 대입과 그 주소의 값을 바꾸는 대입을 구분합니다.

12장 · 움직임

`i*3`을 `i*2`로 바꾸면 반복 횟수는 60회 그대로이고 이동 간격이 줄어듭니다. Sleep만 늘리면 기다리는 시간이 길어집니다. 두 수정을 한 번에 하지 않아야 원인을 비교하기 쉽습니다.

18장 · 모두 켜기

입력 처리에 `if (key=='f') garden_lit=127;`을 추가할 수 있습니다. 화면 안내도 함께 바꾸고 r과 q가 그대로 동작하는지 확인합니다.

18장 · 다섯 등불

등불 수는 5, 허용 키는 1~5, 모두 켜진 상태는 31입니다. 더 일반적으로는 개수가 충분히 작은 정수일 때 `(1<<count)-1`로 마스크를 구할 수 있습니다. 형의 비트 수를 넘기는 시프트는 피합니다.

근거 S04 · 자세한 출처는 책 뒤의 자료 목록

부록 D

8주 수업안

주	수업	집에서 남길 것
1	테리의 헌장 읽기, VM 첫 부팅	프로젝트 목적 세 문장
2	첫 출력, 파일 저장, 변수와 형	바꾼 인사말과 계산 결과
3	조건, 반복, 함수	예상·실제 결과 표
4	배열, 문자열, 주소, 레코드	값과 주소 그림
5	메모리, 파일, 입력, 난수	저장·읽기 확인 기록
6	좌표, 색, 콜백, 애니메이션	색이나 위치를 바꾼 그림
7	키 입력, 소리, DolDoc, 내부 구조	읽어 본 함수 하나의 설명
8	일곱 등불 제작, 검증, 발표	코드·조작법·출처가 있는 작품

한 번의 수업은 설명 15분, 직접 실행 25분, 작은 변형 20분, 서로 설명하기 10분으로 시작할 수 있습니다. 학생의 속도에 맞춰 나누세요. 가상머신 설치에는 별도의 준비 시간이 필요할 수 있습니다.

평가는 외운 문법 수보다 설명 가능한 변화를 봅니다. 무엇을 바꿨는지, 왜 바뀌었는지, 어떻게 확인했는지 말할 수 있다면 좋은 진전입니다.

작품 발표에서는 타인의 고통이나 종교를 흉내 내는 연기를 요구하지 않습니다. 자신이 이해한 기술과 작품에 담은 마음을 직접 설명하도록 합니다.

근거 S01 · S02 · 자세한 출처는 책 뒤의 자료 목록

부록 E

작품을 건네는 짧은 문서

작은 프로그램에도 사용법이 있으면 다음 사람이 쉽게 시작할 수 있습니다. 다음 항목을 자신이 만든 작품에 맞게 채워 보세요.

이름: 작품의 짧은 제목.

무엇을 하는가: 실행하면 보이는 것과 가능한 조작을 두 문장으로 설명합니다.

실행 환경: 원본 TempleOS 5.03인지, 별도 포크인지 적습니다. 사용한 파일 이름과 실행 명령을 넣습니다.

조작과 종료: 사용하는 키와 종료 방법을 함께 씁니다. 저장되는 파일이 있으면 파일명도 적습니다.

만든 부분과 가져온 부분: 테리의 API를 사용했다는 점, 참고한 예제, 자신의 변경을 구분합니다. 다른 작품의 파일을 포함했다면 해당 이용 조건도 확인합니다.

확인한 것: 정상 실행, 경계 입력, 종료, 다시 실행한 결과를 간단히 적습니다. 확인하지 못한 환경은 확인했다고 쓰지 않습니다.

다음 사람이 바꾸기 좋은 곳: 문장, 색, 숫자 하나처럼 작은 출발점을 알려 줍니다. 읽고 고칠 수 있는 선물로 마무리합니다.

이 책의 신규 예제는 함께 제공한 예제 이용 안내에 따라 자유롭게 학습·수정·공유할 수 있습니다. 폰트 프로그램은 배포 파일에 포함하지 않습니다.

근거 S06 · 자세한 출처는 책 뒤의 자료 목록

마침

한 줄을 읽고 한 빛을 남기다

우리는 첫 문장에서 시작해 값과 주소를 배우고, 화면에 빛을 놓았습니다. 마지막의 작은 작품은 많은 기능을 갖고 있지 않습니다. 대신 입력부터 결과까지 자신의 말로 설명할 수 있습니다.

테리의 작업을 따르는 한 가지 길은 그가 남긴 소스를 직접 읽는 것입니다. 또 하나는 우리가 이해한 작은 것을 다음 사람에게 읽을 수 있게 건네는 일입니다. 그 두 가지를 오늘의 코드에서 이어 갈 수 있습니다.

어떤 날에는 한 함수만 이해해도 충분합니다. 어떤 날에는 오류 하나를 찾는 일이 진전입니다. 멈추고 다시 시작할 수 있는 작업 공간을 지키며, 자신의 속도로 계속 만드세요.

오늘의 마지막 기록

내가 이해한 한 줄:

내가 바꾼 한 가지:

다음 사람에게 남길 한 문장:

그 기록과 코드를 함께 보관하세요. 작은 작품을 정성껏 만드는 일이 오래 남기를 바랍니다.

근거 S01 · S02 · 자세한 출처는 책 뒤의 자료 목록

SOURCES / 조사 자료

자료와 근거 1

열람일 2026-11-08. Terry의 문서는 원저자 자료의 보존본이며, 보존 사이트 운영자를 원저자로 표기하지 않았습니다. 소스의 고정 커밋과 파일 해시는 연구 기록에 있습니다.

S01 · TempleOS Charter

Terry A. Davis

헌장 원문 보존본. 신앙적 목표·16색·단순함·공개 소스의 근거.

https://tinkeros.github.io/WbTempleOS/Doc/Charter.html

S02 · Welcome to TempleOS

Terry A. Davis

제작 목적, 명령줄, 함수 실행, 개인 설정과 창 조작.

https://tinkeros.github.io/WbTempleOS/Doc/Welcome.html

S03 · Frequently Asked Questions

Terry A. Davis

그래픽 모드, 컴파일, 메모리, 압축, 네트워크의 설명.

https://tinkeros.github.io/WbTempleOS/Doc/FAQ.html

S04 · HolyC

Terry A. Davis

자료형·기본 인수·연속 비교·함수 주소·연산자 문법.

https://tinkeros.github.io/WbTempleOS/Doc/HolyC.html

S05 · Hello World

Terry A. Davis

첫 출력과 함수 호출의 원본 예시.

https://tinkeros.github.io/WbTempleOS/Doc/HelloWorld.html

SOURCES / 조사 자료

자료와 근거 2

S06 · TempleOS 공식 보존 홈페이지

원저자 Terry A. Davis / 현 보존 관리자 trapexit

1969–2018 추모, 가족의 요청, 보존 자료의 public domain 안내.

https://templeos.org/

S07 · Uses This: Terry Davis

Terry Davis 인터뷰 / Uses This, 2014-05-20

본인의 신앙적 목적, 개발 환경, VM 사용에 대한 직접 인터뷰.

https://usesthis.com/interviews/terry.davis/

S08 · Requirements for TempleOS

Terry A. Davis

x86-64, 최소 512 MiB, VGA, PC 스피커, 장치 제약.

https://tinkeros.github.io/WbTempleOS/Doc/Requirements.html

S09 · Installing TempleOS

Terry A. Davis

Live CD, 마운트, 설치 과정. 빈 실습 디스크와 실제 디스크를 구분.

https://tinkeros.github.io/WbTempleOS/Doc/Install.html

S10 · QEMU System Emulation: Invocation

QEMU 프로젝트

PC 머신, TCG, 메모리, IDE 이미지, 부팅 장치, 네트워크 옵션.

https://www.qemu.org/docs/master/system/invocation.html

SOURCES / 조사 자료

자료와 근거 3

S11 · UTM Documentation: System

UTM 프로젝트

호스트와 게스트 CPU 구조, 에뮬레이션·가상화, i440FX 설정.

https://docs.getutm.app/settings-qemu/system/

S12 · Command Line Overview

Terry A. Davis

Ed, #include, 경로와 드라이브, 파일 실행.

https://tinkeros.github.io/WbTempleOS/Doc/CmdLineOverview.html

---

S13 · Help Index / Key Allocations

Terry A. Davis

Esc 저장, Shift+Esc 취소, F1 도움말, 소스 탐색.

https://tinkeros.github.io/WbTempleOS/Doc/HelpIndex.html

S14 · Memory Overview

Terry A. Davis

작업의 메모리와 코드 힙. 주소 공유에 대한 설명은 FAQ도 참조.

https://tinkeros.github.io/WbTempleOS/Doc/MemOverview.html

S15 · Kernel/KernelC.HH

Terry A. Davis / 소스 보존 저장소

FileRead·FileWrite·GetStr·GetChar·RandU32·Snd 등의 실제 선언.

https://github.com/cia-foundation/TempleOS/blob/c26482bb6ad3f80106d28504ec5db3c6a360732c/Kernel/KernelC.HH

SOURCES / 조사 자료

자료와 근거 4

S16 · RedSea and file utilities

Terry A. Davis

자체 파일시스템. 파일 함수는 S15 선언과 구현을 함께 대조.

https://tinkeros.github.io/WbTempleOS/Doc/RedSea.html

S17 · God / Oracle help

Terry A. Davis

오라클 관련 기능의 길잡이. 종교적 해석은 저자의 주장으로 귀속.

https://tinkeros.github.io/WbTempleOS/LiveHelp/God.html

S18 · Graphics Overview / GrDC.HC

Terry A. Davis

그리기 구조와 컨텍스트. DCAlias·DCFill·DCDel 구현도 대조.

https://tinkeros.github.io/WbTempleOS/Doc/GraphicsOverview.html

S19 · Adam/Gr/GrBitMap.HC

Terry A. Davis / 소스 보존 저장소

GrRect·GrPrint의 실제 인수와 동작.

https://github.com/cia-foundation/TempleOS/blob/c26482bb6ad3f80106d28504ec5db3c6a360732c/Adam/Gr/GrBitMap.HC

S20 · Adam/Gr/GrPrimatives.HC

Terry A. Davis / 소스 보존 저장소

GrLine·GrCircle의 실제 선언과 구현. 원본 철자 유지.

https://github.com/cia-foundation/TempleOS/blob/c26482bb6ad3f80106d28504ec5db3c6a360732c/Adam/Gr/GrPrimatives.HC

SOURCES / 조사 자료

자료와 근거 5

S21 · Scheduler / task functions

Terry A. Davis

협력적 작업 전환. Sleep·Yield는 원본 Kernel 선언 및 FAQ와 대조.

https://github.com/cia-foundation/TempleOS/blob/c26482bb6ad3f80106d28504ec5db3c6a360732c/Kernel/Sched.HC

S22 · Sound / Psalmody

Terry A. Davis

PC 스피커와 음악 도구. 실제 소스 Adam/Snd 및 Apps/Psalmody 참조.

https://tinkeros.github.io/WbTempleOS/Doc/Snd.html

S23 · DolDoc Overview

Terry A. Davis

문서의 서식·위젯·그래픽 자원과 편집 구조.

https://tinkeros.github.io/WbTempleOS/Doc/DolDocOverview.html

S24 · Sprite

Terry A. Davis

스프라이트 편집과 문서 안의 자원.

https://tinkeros.github.io/WbTempleOS/Doc/Sprite.html

S25 · Demo/Graphics/Box.HC

Terry A. Davis / 소스 보존 저장소

3D 선과 행렬, draw_it 콜백을 읽는 후속 실습.

https://github.com/cia-foundation/TempleOS/blob/c26482bb6ad3f80106d28504ec5db3c6a360732c/Demo/Graphics/Box.HC

SOURCES / 조사 자료

자료와 근거 6

S26 · Compiler Overview

Terry A. Davis

컴파일 과정과 원본 컴파일러의 길잡이.

https://tinkeros.github.io/WbTempleOS/Doc/CompilerOverview.html

S27 · ZealOS 프로젝트 README

Zeal Operating System 프로젝트

원본과 구분한 후속 프로젝트. 설명은 2026-11-08 열람 기준.

https://github.com/Zeal-Operating-System/ZealOS

S28 · TempleOS Downloads

TempleOS 보존 사이트

실습에 사용한 TempleOS.ISO의 배포 위치. 다운로드 해시는 별도 검증 기록에 보관.

https://templeos.org/Downloads/

EDITION / 판권과 제작 정보

이 책에 대하여

작은 성전을 짓는 코드
HolyC · TempleOS 처음 배우는 강의 책
한국어 학습판 · 2026년 11월 8일

원 운영체제·언어·기술 문서: Terry A. Davis
제작 요청: 박성모
본 학습판의 한국어 설명과 실습 예제는 새로 작성했습니다. 원저자의 공식 교재나 저자 승인판으로 표시하지 않습니다.

원본 TempleOS 자료의 public domain 안내는 공식 보존 사이트를 따릅니다. 후속 프로젝트와 외부 문서의 이용 조건은 각 원출처를 따릅니다. 이 책의 실습 파일은 학습하고 수정할 수 있도록 함께 제공합니다.

서체: 윤모아이, 윤고딕 700, 윤픽셀, 윈도우고딕, Menlo. 사용자의 로컬 활성 서체로 조판했으며 폰트 프로그램을 배포 패키지에 포함하지 않습니다. 편집 원본을 열 때는 해당 서체의 이용 권한과 설치가 필요합니다.

본 학습판의 코드 실행·지면·원본 대조 결과는 함께 제공하는 검증 기록에서 확인할 수 있습니다. 사용자가 확인한 환경과 제작 중 확인한 환경을 구분해 기록합니다.

읽기	만들기	나누기
원문과 코드를 이해하기	한 가지를 바꿔 확인하기	다음 사람이 읽을 수 있게